Enhancing the Efficacy of Ransomware Detection Systems Using the Random Forest Algorithm and Implementing Real-Time Endpoint Protection.

xxxxxxxxxxxx
xxxxxxxxx

Supervisor: Dr. Althaff Irfan Cader Mohideen

Thesis for the partial fulfilment
of the requirements for the Master's Degree
In Cyber Security

University of the West of Scotland
School of Computing, Engineering and Physical Sciences
22/08/2025

SAMPLE THESIS

To My Loving Family.....

SAMPLE THESIS

UWS UNIVERSITY OF THE WEST of SCOTLAND

DECLARATION OF ORIGINALITY

I declare that this is an original study based on my own work, and that I have not submitted it for any other course or degree.

Signature: xxxxxxxxxx

Banner ID: xxxxxxxxxxxxxxxxxx

Date: 22/08/2025

SAMPLE THESIS

UWS UNIVERSITY OF THE WEST of SCOTLAND

LIBRARY RELEASE FORM

Surname: xxxxxxxx	
First Name: xxxxxxxx	**Initials:** xxxxxx
Banner No: xxxxxxxx	
Course Code: COMP11024	
Course Title: MSc Advanced Computer Security	
Project Supervisor: Dr. Althaff Irfan Cader Mohideen	
Project Title: ***Enhancing the Efficacy of Ransomware Detection Systems Using the Random Forest Algorithm and Implementing Real-Time Endpoint Protection.***	
Session: 2024/2025	**Date of Submission:** 22/08/2025
Signature: xxxxxxxxxx	

SAMPLE THESIS

ABSTRACT

Since the creation of ransomware, it has emerged as one of the most disruptive and damaging forms of cyber-attacks, encrypting user data and applications and then demanding ransom to regain access. Traditional signature-based methods struggle against newer and zero-day variants of ransomware, which makes it evident that the need for more accurate and adaptable detection systems is necessary. This project will tackle these issues by designing and implementing a Machine Learning model with increased effectiveness for detecting ransomware, using the Random Forest algorithm for classification.

Development began with a structured dataset containing 21,752 benign and malicious samples, which was cleaned, pre-processed, and then augmented to include 86,006 samples. The data augmentation involved duplicating the original dataset and applying Gaussian noise to the existing samples, thereby increasing variety and preserving integrity to the new samples. The Random Forest model was then trained and tested using both datasets with different seeds to evaluate accuracy through different runs. It was finally implemented and tested to simulate real-time detection to understand its application in real-world environments.

Testing and results demonstrated that the model achieved a strong baseline accuracy of 99.2-99.6% on the original dataset, which was improved to 99.8-99.95% after augmentation. Precision, recall, and F1-scores also achieved results of 0.99-1.00 consistently, supported by balanced confusion matrices and minimal misclassifications. The results of the implemented simulation also supported the model with 99.5% correctly predicted samples before augmentation and 99.95% after.

This project provides evidence that the Random Forest algorithm, combined with an augmented dataset, can provide a highly accurate and effective detection solution. The key findings highlight the importance of data diversity in machine learning models within cybersecurity.

Keywords: Ransomware Detection, Machine Learning, Random Forest, Dataset Augmentation, Real-Time Simulation, Cybersecurity.

SAMPLE THESIS

ACKNOWLEDGEMENTS

The completion and success of this project would not have been achieved without the support and advice of specific individuals. Therefore, I would like to take the opportunity to thank and show my appreciation to all those individuals who helped and supported me.

- First and foremost, I would like to thank Dr. Althaff Irfan Cader Mohideen for supervising this project and for the fantastic support, guidance, and constructive feedback I have received throughout it.
- Dr. Raman Singh for approving my project, acting as the moderator and supporting my project.
- Ms. Daune West for the support throughout the Research Methods and Design and Master's Project modules.
- Special thank you to the Student Hub and Ewan Loughlin of the UWS IT department in Lanarkshire for helping me regain access to my accounts during a stressful time.
- All my friends who have communicated and supported me through my project.
- Last, but not least, my parents, who have immensely supported me throughout my university journey and have always offered support and guidance.

SAMPLE THESIS

TABLE OF CONTENTS

Chapter 1 - Introduction 13
1.1. Chapter Overview 14
1.2. Justification 14
1.3. Background 14
1.4. Aim 16
1.5. Objectives 16
1.6. Structure of the Report 17
1.7. Chapter Summary 17
Chapter 2 - Literature Review 19
2.1. Chapter Overview 20
2.2. Ransomware Background 20
2.3. Ransomware Types and Encryptions 21
2.3.1. Ransomware Types 21
2.3.2. Ransomware Families 22
2.3.3. Ransomware Encryption 23
2.4. Traditional Detection Methods 25
2.4.1. Signature-based Detection 25
2.4.2. Heuristic-based Detection 26
2.5. Machine Learning Models 27
2.6. Machine Learning Algorithms 28
2.7. Studies with Random Forest Algorithm 29
2.8. Real-Time Endpoint Protection 30
2.9. Gaps in Literature 31
2.10. Chapter Summary 31
Chapter 3 - Project Management 32
3.1. Chapter Overview 33
3.2. Project Plan 33
3.2.1. Project Specification Form 33
3.2.2. Gantt Chart 34
3.3. Monitoring 35
3.4. Risk Assessment 35
3.5. Chapter Summary 36
Chapter 4 - Research Design and Methodology 37
4.1. Chapter Overview 38

SAMPLE THESIS

4.2. Dataset Description 38
4.3. Data Preprocessing 39
4.3.1. Data Cleaning 39
4.3.2. Label Encoding 40
4.3.3. Hexadecimal to Integer Conversion 40
4.3.4. Category Feature Encoding 40
4.3.5. Train-Test Splitting 40
4.3.6. Cleaned Dataset 41
4.4. Model Training/Development 41
4.4.1. Model Selection 41
4.4.2. Model Training 41
4.4.3. Prediction and Evaluation 41
4.5. 1st Testing Phase 42
4.6. Dataset Augmentation Methodology 42
4.6.1. Selected Columns for Augmentation 42
4.6.2. Create Duplicate Copies 42
4.6.3. Applying Gaussian Noise 42
4.6.4. Merging Augmented Datasets 42
4.7. 2nd Testing Phase 42
4.8. Real-Time Detection 43
4.8.1. Real-Time Detection Code 43
4.9. 3rd Testing Phase 43
4.10. Software Used and Python Libraries 43
4.11. Chapter Summary 43
Chapter 5 - Design 45
5.1. Chapter Overview 46
5.2. Project Design Methodology 46
5.3. Overall Project Design 46
5.3.1. Waterfall Diagram of Project 47
5.3.2. Overall Design Discussion 47
5.4. Dataset Design 48
5.4.1. Dataset Cleaning Diagram 48
5.4.2. Dataset Cleaning Process 48
5.5. Dataset Augmentation Design 51
5.6. Random Forest Classification 54

SAMPLE THESIS

5.7. Chapter Summary 55
Chapter 6 - Implementation 56
6. Chapter Overview 57
6.1. Real-Time Detection Simulation 57
6.2. Chapter Summary 58
Chapter 7 - Testing 59
7.1. Chapter Overview 60
7.2. Indicators of Test Success 60
7.2.1. Pass Criteria 60
7.2.2. Fail Criteria 60
7.3. Test Schedule 60
7.4. Original Dataset and Classification Testing 61
7.5. Augmented Dataset and Classification Testing 63
7.6. Real-Time Detection Simulation and Testing 64
7.7. Chapter Summary 66
Chapter 8 - Results 67
8.1. Chapter Overview 68
8.2. Precision, Recall and F1 Score Results 68
8.3. Accuracy Trend Line Graph 71
8.4. Accuracy and Seed Comparison 72
8.5. Confusion Matrix Heatmaps 73
8.6. Correctly Predicted Sample Charts 75
8.7. Chapter Summary 76
Chapter 9 - Conclusion and Recommendations 77
9.1. Chapter Overview 78
9.2. Key Achievements 78
9.3. Limitations in the Current Model 78
9.4. Challenges and Issues During Project 79
9.5. Use of University and Course Materials 80
9.6. Future Recommendations 80
9.8. Chapter Summary 81
Chapter 10 - Self-Evaluation 82
10.1. Chapter Overview 83
10.2. Technical Skill Development 83
10.3. Project Management and Time Management 83

SAMPLE THESIS

10.5. Communications and Feedback 84
10.6. Personal Development 84
10.7. Areas for Improvement 85
10.8. Chapter Summary 85
Chapter 11 - References 86
Chapter 12 - Bibliography 91
Chapter 13 - Appendix 94
13.1. Project Specification Form 95

SAMPLE THESIS

LIST OF FIGURES

Figure 1.1: The process of a ransomware attack requesting crypto payment. (Yan and Khoei, 2025).

Figure 2.1: Total value of money received by ransomware attackers 2019 through 2023 (Chainalysis, 2024)

Figure 2.2: All top 10 most active ransomware families and the number of attacks completed in 2024 (Adi Bleih, 2025).

Figure 2.3: Diagram of the symmetric encryption process.

Figure 2.4: Diagram of the asymmetric encryption process (AppView, 2025)

Figure 2.5: Diagram of the structure of a signature-based detection system (Albishry et al., 2022)

Figure 2.6: Overview of detection techniques' advantages and disadvantages (Yan and Khoei, 2025).

Figure 2.7: Examples of various Machine Learning models (Sarker, 2021)

Figure 3.1: Gantt chart of weekly progress and plan

Figure 5.1: Waterfall diagram showing each main component of the project. Created by Kai Johnson using Creately.

Figure 5.2: Diagram showing each component of the dataset cleaning stage. Created by Kai Johnson using Microsoft Word.

Figure 5.3: Code snippet used to import libraries, read the dataset for cleaning and columns being dropped.

Figure 5.4: Code snippet of class labels being changed to binary format.

Figure 5.5: Code snippet of converting hexadecimal strings to integers.

Figure 5.6: Code snippet of hot encoding categorical features.

Figure 5.7: Original dataset before cleaning.

Figure 5.8: New dataset after cleaning.

Figure 5.9: Code snippet for selecting data for data augmentation.

Figure 5.10: Code snippet for altering data for data augmentation.

Figure 5.11: Code snippet for merging newly created samples and the original dataset.

Figure 5.12: image showing the completion of augmentation with the old and new sizes of the dataset.

Figure 5.13: Before dataset augmentation

Figure 5.14: after dataset augmentation.

SAMPLE THESIS

Figure 5.15: Code snippet for label and feature separation.

Figure 5.16: Code snippet for train and test splitting.

Figure 5.17: code snippet for Model training and evaluations.

Figure 6.1: Code snippet for model simulation of real-time detection.

Figure 6.2: Code snippet for model simulation of real-time detection.

Figure 7.1: Code output of results for Test ID 13 with a random state seed of 40, including accuracy, precision, recall, F1 score, and confusion matrix.

Figure 7.2: Code output of results for Test ID 11 with a random state seed of 35, including accuracy, precision, recall, F1 score, and confusion matrix.

Figure 8.1: Graph illustrating the precision, recall, and F1 scores for both benign and malware classes, using the best tested seed before augmentation.

Figure 8.2: Graph illustrating the precision, recall, and F1 scores for both benign and malware classes, using the best tested seed after augmentation.

Figure 8.3: Accuracy trend before and after augmentation across all 20 tested seeds.

Figure 8.4: Accuracy comparison before and after augmentation across all 20 tested seeds.

Figure 8.5: Confusion matrix heatmap before dataset augmentation.

Figure 8.6: Accuracy comparison before and after augmentation across all 20 tested seeds.

Figure 8.7: Pie chart showing correctly predicted samples versus incorrect samples before dataset augmentation.

Figure 8.8: Pie chart showing correctly predicted samples versus incorrect samples after dataset augmentation.

SAMPLE THESIS

LIST OF TABLES

Table 3.1. A detailed risk assessment of potential risks that could occur throughout the project, along with the mitigation measures in place to prevent any issues.

Table 4.1: Descriptive table showing the dataset before augmentation with all cyber families, types of malware, and total numbers

Table 7.1. *Test schedule showing dates, times, and any comments noted*

Table 7.2*: Accuracy results for the classification on the dataset before augmentation, including true positives, true negatives, false positives, and false negatives.*

Table 7.3*: Accuracy results for the classification on the dataset after augmentation, including true positives, true negatives, false positives, and false negatives.*

Table 7.4: *Prediction results for each sample run on test 1, including their classification predictions and actual classifications.*

Table 7.5*: Prediction results for each sample run on test 2, including their classification predictions and actual classifications.*

Table 7.6*: Prediction results for each sample run on test 3, including their classification predictions and actual classifications.*

Table 7.7*: Prediction results for each sample run on test 4, including their classification predictions and actual classifications.*

Table 7.8*: Prediction results for each sample run on test 5, including their classification predictions and actual classifications.*

SAMPLE THESIS

Chapter 1

Introduction

SAMPLE THESIS

1.1. Chapter Overview

This chapter explores the background of ransomware and existing detection systems. It is structured to highlight the current issues with traditional detection methods for ransomware and the need for new, improved solutions to enhance security, along with an overview of a proposed solution. Furthermore, this chapter discusses the project's aims and objectives in developing this solution, as well as the motivation behind undertaking this project.

1.2. Justification

Ransomware attacks are increasing rapidly each year, targeting various individuals, businesses, and government organisations, and costing billions in damages, a figure that rises annually. Due to the swift growth of ransomware attacks, traditional signature-based detection systems struggle to keep up with these evolving threats. While relying on known malware signatures, Machine Learning offers a more dynamic alternative, capable of identifying suspicious trends in computer behaviour. The Random Forest algorithm shows promise due to its robustness and high accuracy in detection and binary classification tasks, such as malware detection. In light of the rapid growth of ransomware attacks, this research addresses a critical security need by investigating a Machine Learning-based detection system and exploring its application in real-time endpoint protection to enhance computer security.

1.3. Background

Ransomware attacks involve deploying malicious software or code that encrypts the system's data or files, restricting access to the computer’s data or system until a ransom is paid. These attacks will typically occur when an unsuspecting individual or organisation receives an infected email attachment and inadvertently activates a malicious link that facilitates the installation of the ransomware (Yan and Khoei, 2025; Reshmi, 2021). Once this malware is activated on the individual's device, the system encrypts all files, thereby rendering them inaccessible. When all files have been encrypted, the attackers responsible for the incident demand a ransom payment in exchange for the decryption key, which will restore access to the files. In most instances, the ransom payment is requested to be made in cryptocurrencies such as Bitcoin or Litecoin, due to the difficulty in tracing these forms of payment back to the

SAMPLE THESIS

attackers (Yan and Khoei, 2025; Beaman, C., Barkworth, A., Toluwalope, A., Hakak, S., and Khan, K., 2021).

Figure 1.1: The process of a ransomware attack requesting crypto payment. (Yan and Khoei, 2025).

Recently acquired data reflects the growing scale of the problem, as a report completed by Adi Bleih (2025) indicates that 5,414 ransomware attacks occurred against various organisations globally in 2024, representing an 11% increase compared to 2023. Ransomware incidents are typically documented on a quarterly basis; nevertheless, in 2024, a notable decline in attacks was observed during the first quarter. In contrast, the second quarter marked the beginning of a worrying escalation in attacks, which continued throughout the year. The fourth quarter, which saw 1,827 attacks, represented 33% of the total and experienced a significant increase, making it the most active period of the year (Adi Bleih, 2025).

Numerous software companies have developed methods for identifying ransomware attacks. One such method is signature-based software, which represents a more conventional approach to combating ransomware and has proven effective in identifying potential threats. However, in the context of unknown attacks, also referred to as zero-day attacks, signature-based software is inadequate, as it fails to recognise newly created malicious files (Yeap, 2024; Alraizza and Algarni, 2023).

As traditional methods are increasingly circumvented and exploited by new and emerging malware threats, innovative approaches, such as Machine Learning (ML), demonstrate significant potential for malware detection. In the information technology industry, the majority of cybersecurity experts mention that malware detection software and systems driven by ML algorithms will serve as a vital solution for identifying and mitigating malware attacks (Saad, Briguglio and Elmiligi, 2019). Numerous justifications underpin the utilisation of ML as an advanced solution for malware detection, including its capacity to learn from data and identify unknown or zero-day malware, its adaptability to evolving attack strategies, and its proficiency in recognising patterns within extensive datasets.

SAMPLE THESIS

1.4. Aim

In light of the dynamic nature of ransomware and the imperative for more sophisticated detection methodologies, this research intends to design, develop, and evaluate a ransomware detection system employing the Random Forest algorithm to enhance its efficacy for real-time endpoint protection, thereby improving detection. Additionally, it aims to assess the Machine Learning model's ability to distinguish between malicious and benign samples, as well as its potential application in an operational security framework. This project aims to provide and contribute to the existing body of research aimed at improving intelligent threat detection systems by offering a functional detection system characterised by high detection accuracy and real-time responsiveness.

1.5. Objectives

To complete this research, several objectives will be pursued to aid in the successful development of an improved ransomware detection method using the Random Forest algorithm. The objectives are as follows

- The creation of a literature review of existing ransomware detection systems that employ Machine Learning or artificial intelligence methodologies, the Random Forest algorithm, and real-time endpoint protection techniques.
    1. Background of ransomware.
    2. Study of current traditional ransomware detection techniques.
    3. In-depth study on newer Machine Learning techniques.
    4. Identify current techniques using Random Forest algorithms.
    5. Study and explain real-time endpoint protection.
    6. Explain the role of this project.
- Distinguish and gather a relevant dataset for the Machine Learning model, which will include infected ransomware files and benign samples. Preprocess and clean the dataset to ensure compatibility with the Machine Learning model and algorithm.
- A Machine Learning model will then be developed utilising PyCharm, implementing the Random Forest algorithm through Python-based tools, including scikit-learn.

SAMPLE THESIS

- The performance of the model will be evaluated using several metrics, such as accuracy, precision and F1 scores. Additionally, the applicability of the Machine Learning model in a virtual real-time simulation to understand how the model will work in a real-time environment.
- Recommendations for future work, including system integration and scalability, will also be provided. Finally, a comprehensive final report will be prepared to present all recorded findings, results, and conclusions derived from the completed work.

1.6. Structure of the Report

The report for this project will be organised in a clear and cohesive format for each chapter to ensure it is easy to read. The format of each chapter is as follows:

- Introduction: Offers information about the overall project, including background details and issues related to ransomware detection.
- Literature Review: Provides insight into current research on ransomware detection and current methods using the proposed model algorithm.
- Research Design/Methodology: detailed processes of data preprocessing, model design and training, real-time implementation, and hyperparameter tuning.
- Testing and implementation: Evaluation of the working model and its deployment.
- Evaluation: Evaluation of concluded results and detailed analysis of precision, accuracy, and classifications.
- Conclusion: summarise the project, outline future work, and discuss any complications.
- References/Appendices: List of all utilised references and code employed for the completed project.

1.7. Chapter Summary

This chapter discusses the threat of ransomware and the background of traditional detection methods, such as signature-based detection, which cannot identify new and evolving attacks. It introduces Machine Learning, specifically the Random Forest algorithm, as a more effective approach to detecting ransomware. The chapter also introduces the project's aim to develop and evaluate a detection system that can

SAMPLE THESIS

differentiate between benign and malicious samples, as outlined in the objectives. Lastly, the structure of the report is also outlined to show the flow and what to expect.

The next chapter will be the literature review, which documents research findings on the current topic and problem mentioned in the introduction chapter.

SAMPLE THESIS

Chapter 2

Literature Review

SAMPLE THESIS

2.1. Chapter Overview

The previous chapter provided an overview and background of the project, including details on its aims, objectives, and justification.

This literature offers a critical analysis of the current research on ransomware, as well as detection and prevention systems. It specifically emphasises the role of Machine Learning techniques, including Random Forest, and real-time endpoint protection to enhance security. The review aims to offer insights into current identification methods for ransomware, identify their limitations, and highlight areas that may require further research.

2.2. Ransomware Background

Throughout the history of the internet, numerous advantages have been established for both legitimate users and cyber criminals. While technological advancements continue to progress, many crime schemes remain unchanged. Every day, criminal activities include extortion, blackmail, and theft. Although these offences are not novel, they have now been enhanced by automation, enabling cyber-attacks to be executed automatically (O'Kane, Sezer and Carlin, 2018). One of the predominant forms of cyberattacks is ransomware, which traces its origins to the initial model developed in 1989, known as 'AIDS.' This model was deployed as a Trojan horse, which was utilised through a floppy disk, a technological innovation considered state-of-the-art in 1989 (O'Kane, Sezer, and Carlin, 2018). Since the creation of the first ransomware, its growth has been measured at over 600%, with new families, such as Cerber, Locky, WannaCry, and CryptoWall, emerging. A prevalent form of ransomware is crypto-ransomware, which continues to pose a significant threat. This type aims to encrypt all important information and data in the system, such as documents, images, and videos, without compromising the system's functionality. More modern methods, such as Ransomware as a Service (RaaS) have now been created, which is a distribution service and can allow hackers that have made ransomware to lease the software out to other cybercriminals or individuals who lack hacking skills (Razaulla, Fachkha, Markarian, Gawanmeh, Mansoor, Fung, and Assi, 2023; O'Kane, Sezer, and Carlin, 2018). Cybercriminals employ numerous techniques to compromise individuals' systems. For instance, Trojans, which resemble legitimate software, are malicious code disguised as benign applications that embed themselves within the system.

SAMPLE THESIS

Another infection technique is a waterhole attack, which occurs when attackers hack busy websites and implant their malicious code in specific download links that activate when an unsuspecting individual clicks to download the software. A social engineering technique known as phishing is another commonly employed method that exploits online users by mimicking individuals to induce them to click on or download malicious software (O'Kane, Sezer, and Carlin, 2018). Due to these infection techniques, large numbers of ransomware incidents occur, causing severe impacts on organisations and individuals. Several impacts can affect organisations after a ransomware attack, including financial implications, reputational damage, data loss, and revenue loss (GOV, 2025; JPMorgan, 2023). Due to this, many companies may need to employ quality ransomware detection systems, as the sophistication of ransomware may make it tricky to identify newer techniques.

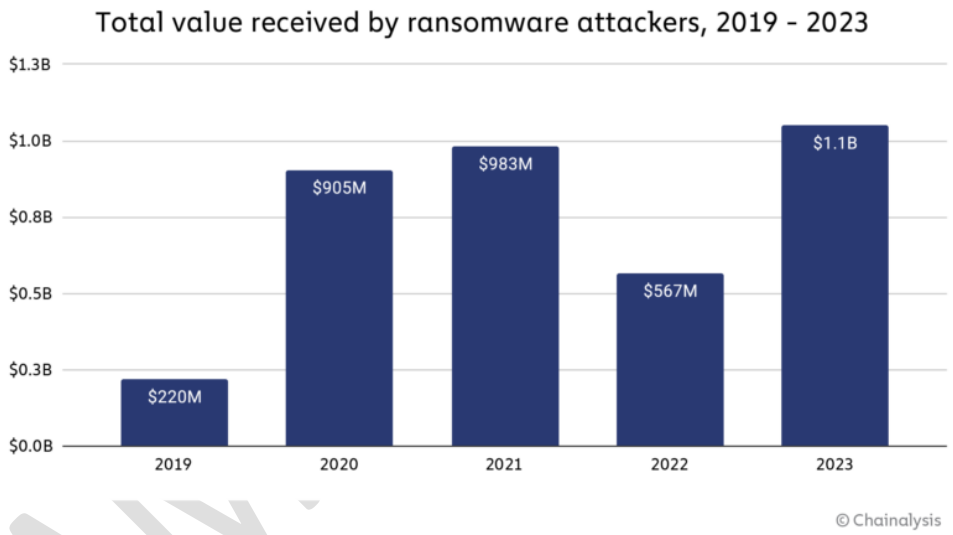

Figure 2.1: Total value of money received by ransomware attackers 2019 through 2023 (Chainalysis, 2024)

2.3. Ransomware Types and Encryptions

2.3.1. Ransomware Types

Due to the rapid increase in ransomware attacks over the years, various types have emerged, each acting differently but with the same ultimate goal of requesting a ransom payment. The most commonly used ransomware types include crypto-ransomware, also known as encryptors, which are the well-known and damaging variants of ransomware, as they encrypt all data on a system and make it completely

inaccessible without any decryption keys (Baker, 2025). Another type is locker ransomware, which locks you entirely out of any system it has infected, ultimately making the files and applications on that system inaccessible. With the malware on the system, it will display a ransom that the victim must pay to restore access to the system, and it may include countdown clocks to emphasise the urgency to the victim, warning of potential loss of files and applications (Baker, 2025; SentinelOne, 2025). Leakware, another commonly used ransomware attack, is a technique that threatens to leak sensitive company information online. This causes many organisations to panic and ultimately pay a ransom to prevent any data from being exposed in the online and public domains. A popular variant of this is police-themed ransomware, which impersonates law enforcement and threatens that illegal online activity has been detected, and jail time can be avoided if the company or individual pays a fine (Baker, 2025). Lastly, it is scareware, which is a type of software that impersonates a virus detection application, informing the individual that malware has been detected on their system and requesting payment to resolve the issue. When this software is applied to the system, it can lock individuals out or flood the system screens with advertisements and pop-up alerts, without damaging any files (Baker, 2025; SentinelOne, 2025). Although many ransomware types are used in attacks, certain ransomware families employ different techniques and modify their code and software to suit their specific needs better.

2.3.2. Ransomware Families

The majority of ransomware attacks that occur are not random; they are coordinated attacks that have been planned against specific and targeted organisations. All ransomware attacks are planned and executed using particular styles and patterns. These codes can then be linked and grouped into different ransomware families based on their signature codes, malicious commands, and attack styles (Bhushan, 2024). The attack styles help identify the ransomware family, which assists in tracking their activities. Additionally, grouping attacks by family aids in defence and provides a better understanding of how their attacks work, allowing for the prevention of future attacks and the implementation of preventative measures to protect organisations (Bhushan, 2024).

SAMPLE THESIS

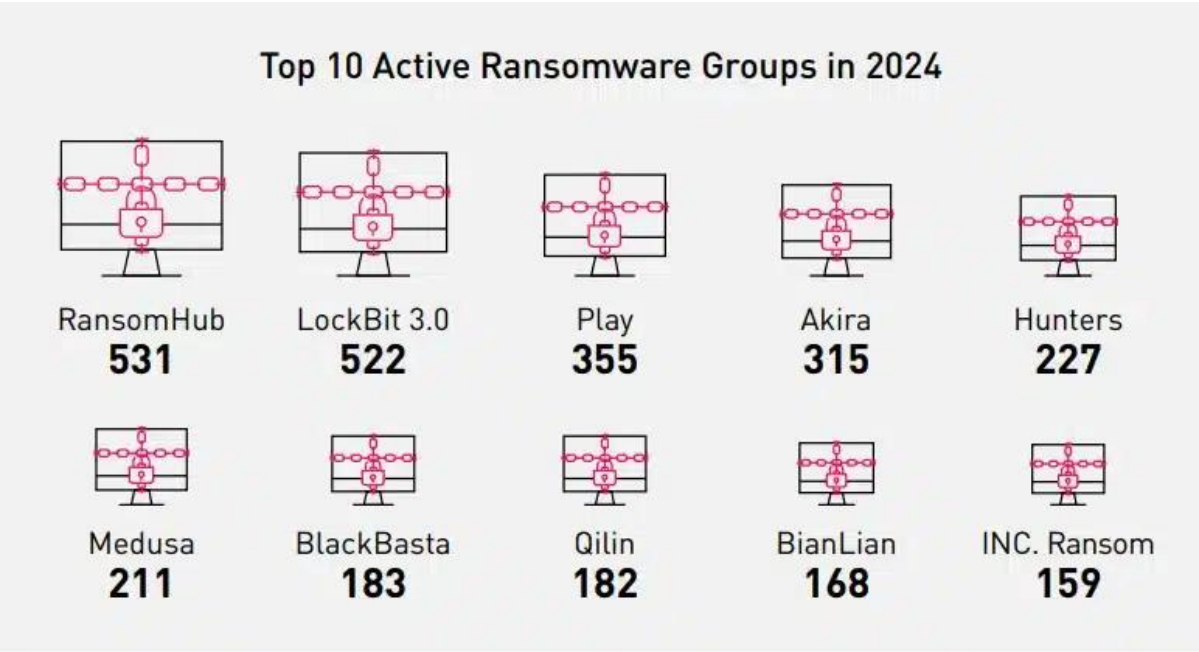

*Figure 2.2: All top 10 most active ransomware families and the number of attacks completed in 2024 (*Adi Bleih, 2025).

Several ransomware families have been identified in 2024, but the most destructive and prevalent is RansomHub, with a total of 531 attacks on organisations. Although RansomHub is the most dangerous in terms of attack rate, according to research by Adi Bleih (2025), in August 2024, 160 out of 190 organisations chose not to pay. However, according to the admins of RansomHub, the main goal is not on payment rates but the volume of attacks made, which will lead to increased revenue. The number of attacks depends on the successful encryption of data, with different malware employing various encryption methods.

2.3.3. Ransomware Encryption

When ransomware is installed on a system, it will encrypt any important files and data on the targeted system. To do this, it will use cryptographic techniques to encrypt the data. There are three commonly used encryption techniques in ransomware: symmetric encryption, client asymmetric encryption, and server asymmetric encryption (Murphy, 2023). With the symmetric encryption technique, it can encrypt various large files quickly and efficiently. After all ransoms have been paid, the attacker can then distribute the public key to the victim to decrypt the files stored on the drives. Some of the older versions of ransomware utilise the AES encryption algorithm. However, with these simple techniques, researchers can identify methods to find the

keys used to decrypt data, which provides opportunities to develop a defence tool to decrypt the data (Murphy, 2023).

Figure 2.3: Diagram of the symmetric encryption process (Murphy, 2023)

Another technique is client asymmetric encryption, in which the ransomware creates a pair of public and private asymmetric keys. The public key is used to encrypt all the files, and the private key is stored on the server to maintain its security. However, this type of encryption method has some disadvantages, such as slow encryption times for larger files, which makes it difficult for the ransomware to complete its task. Additionally, both the infected computer and the attacker's computer must be connected to the internet; if either loses connection, the encryption process will stop (Murphy, 2023).

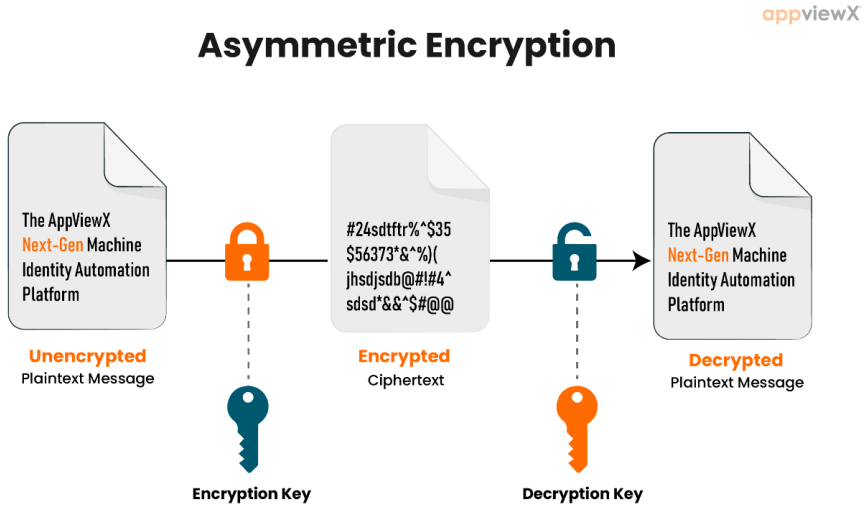

Figure 2.4: Diagram of the asymmetric encryption process (AppView, 2025)

Lastly, there is server asymmetric encryption. This technique involves the server generating two keys, with the primary public key encoded into the ransomware's program. Each file on the system will then be encrypted with the generated public key created by the server, and only the private key can decrypt the files (Murphy, 2023). Due to these ransomware techniques and encryption processes, many companies may need to employ quality ransomware detection systems, as the sophistication of ransomware can make it challenging to identify.

2.4. Traditional Detection Methods

2.4.1. Signature-based Detection

Given the increasing incidence of ransomware attacks, various traditional detection methods have been developed to protect organisations. The most common conventional method is signature-based detection, which employs a series of technical processes to accurately and precisely identify ransomware (Yan and Khoei, 2025; Alraizza and Algarni, 2023). The primary component of this technique is the use of signatures, which are unique digital patterns, such as binary code and hashes. When attacks occur, the signatures of these files are stored in datasets containing other previously identified ransomware files, which the signature-based software can use to analyse and differentiate between malicious and benign files before they are introduced into the system (Yan and Khoei, 2025). Nevertheless, the traditional signature-based detection approach presents certain limitations, particularly its difficulty in identifying unknown threats, commonly known as zero-day attacks, as the signature-based detection does not recognise newly created malicious files (Yeap, 2024; Alraizza and Algarni, 2023).

SAMPLE THESIS

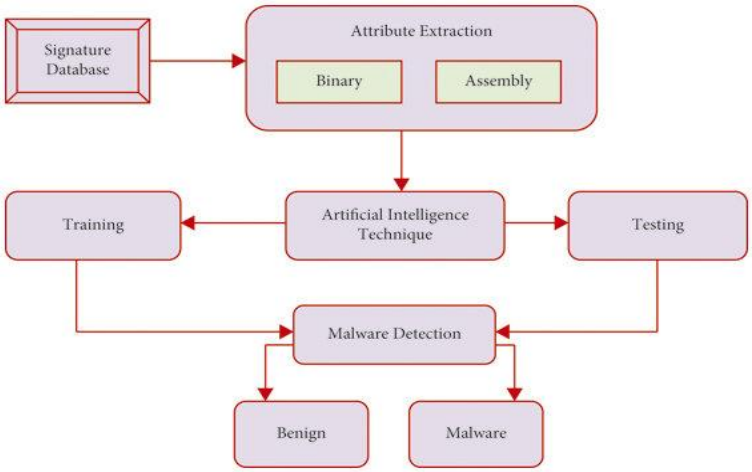

Figure 2.5: Diagram of the structure of a signature-based detection system (Albishry et al., 2022)

2.4.2. Heuristic-based Detection

Another detection technique involves heuristic-based detection, which offers greater flexibility and adaptability compared to signature-based methods. While signature-based techniques rely on known identifiers of various malware, heuristic detection provides an alternative approach by assessing behavioural patterns and anomalies indicative of system infection, thereby offering a more dynamic detection method (Yan and Khoei, 2025). This technique's components are based on establishing specific rules or heuristics that emulate ransomware behaviour, followed by monitoring systems and networks for anomalies that these rules may flag. If any abnormalities are detected and flagged as malicious by the created rules, there could potentially be ransomware within these files. Then the appropriate action will be taken to prevent any installations on the systems (Alraizza and Algarni, 2023).

One key advantage of this technique is its ability to identify newer and unknown variants of ransomware that don’t match other existing signatures or known patterns. In addition, it provides fewer false positives compared to signature-based detection, as it focuses on the behaviour of the ransomware rather than code patterns. However, heuristic-based detection possesses certain limitations, as the technique depends on predefined and established rules that may not encompass all ransomware behaviours. Furthermore, attackers could circumvent this method by modifying and altering the ransomware's behaviour to evade detection (Alraizza and Algarni, 2023).

SAMPLE THESIS

Techniques	Advantages	Disadvantages
Reliable Protocols and Standards	Security foundation via protocols like SSL, TLS Adapts to evolving threats with regular updates	Resource-intensive with continuous monitoring May not effectively counter complex threats alone
Education and Training	Reduces human error by enhancing cybersecurity knowledge Tailored to specific organizational roles	Requires ongoing resource commitment Effectiveness depends on employee engagement
Security Policy	Provides clear IT asset management guidelines Facilitates a structured response to incidents	Needs regular updates and organizational support May be viewed as restrictive by employees
Software Prevention	Blocks threats with anti-ransomware and antivirus solutions Adapts to new threats with updates	Requires consistent updates and correct configuration Potential for false positives
Encryption and Authentication	Ensures data confidentiality and integrity Restricts access to authorized users	Complex implementation and management Requires careful key and security practices
Intrusion Prevention	Enhances security by automatically detecting and blocking threats Tailorable to specific network needs	Needs regular updates to maintain effectiveness Significant resource requirements for maintenance
Protection Tools, Frameworks, and Software	Reduces attack impact with systems like intrusion detection Includes data backups for recovery	Primarily reactive, not preventive Dependent on the effectiveness of other security measures

Figure 2.6: Overview of detection techniques' advantages and disadvantages (Yan and Khoei, 2025).

The primary objective of ransomware detection is to ensure swift identification, avert system harm, and reduce the incidence of false positives. Owing to the limitations inherent in conventional detection methods, researchers have been exploring the application of emerging techniques such as Machine Learning (ML), which present a promising alternative by overcoming many of the shortcomings associated with traditional approaches.

2.5. Machine Learning Models

Several advancements in ransomware detection have been introduced, including Machine Learning (ML), which offers a more scalable and adaptive solution compared to traditional detection methods. ML analyses system and application behaviour using various algorithms capable of identifying significant patterns or anomalies within extensive datasets that may be too complex for the human eye to detect manually (Masum, Hossain, Shahriar, Qian, Lo, and Adnan, 2022; Alraizza and Algarni, 2023). The algorithms employed in ML models can be calibrated and trained with datasets containing ransomware-infected files and benign samples, enabling the models to learn and distinguish between malicious and non-malicious activity (Alraizza and Algarni, 2023).

SAMPLE THESIS

ML offers numerous advantages in ransomware detection; however, its primary benefit is the capacity to identify previously unknown ransomware files, including zero-day attacks, by learning distinct behavioural patterns. Furthermore, Machine Learning models can continually adapt and learn in response to evolving threats, thereby enhancing detection accuracy over time and reducing false positives by accurately distinguishing between benign and malicious behaviours (Rele, Samuel, Patil, Krishnan, 2025).

Several Machine Learning techniques are frequently employed for detecting ransomware. The primary approach among these is supervised learning, where the model's objective is to learn functions that map input data to labelled outputs. This task encompasses classification and regression tasks, where classification separates and categorises data, and regression tasks predict and fit the data (Sarker, 2021; Belcic and Stryker, 2021).

Additionally, unsupervised learning is another technique that focuses on analysing unlabelled data with no human intervention, making it a data-driven approach. It is mainly used for discovering hidden patterns, anomaly detection, clustering data, and exploratory analysis (Sarker, 2021; IBM, 2021). Lastly, semi-supervised learning combines the first two methods by utilising both labelled and unlabelled data. This approach is especially significant for ransomware detection, as labelled data can be scarce, whereas unlabelled data is more plentiful. Therefore, semi-supervised learning enables efficient model training with limited labelled datasets (Sarker, 2021; IBM, 2023).

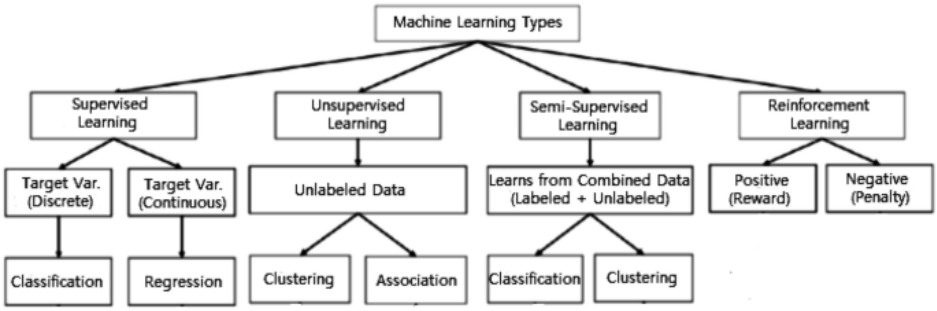

Figure 2.7: Examples of various Machine Learning models (Sarker, 2021)

2.6. Machine Learning Algorithms

Within ML, various algorithms are used to detect ransomware across different models. For example, a commonly employed algorithm, Random Forest (RF), performs

strongly in ransomware detection. As RF is an efficient learning technique, it can combine the predictions of multiple decision trees to improve its reliability and accuracy (Reddy, Kumar, Siva, Pothuraju, 2023). The method employed is known as bootstrap sampling, which produces multiple training subsets utilised to construct each decision tree. RF is more robust and possesses enhanced generalisation capabilities, as traditional decision trees evaluate every node, whereas RF considers a randomly selected subset of features (Yüksel, 2025). The advantages of the Random Forest algorithm lie in the decision outputs of its constituent decision trees, which serve to reduce prediction errors and mitigate significant overfitting, particularly when used with high-dimensional and extensive malware datasets. The key advantage of RF is its ability to achieve high accuracy and scalability. However, it also presents certain disadvantages, such as its requirement for substantial computational resources and prolonged training durations, particularly when a large number of decision trees and hyperparameter tuning are involved (Yüksel, 2025).

2.7. Studies with Random Forest Algorithm

Several studies have been completed to demonstrate the effectiveness of the Random Forest algorithm for malware detection. The algorithm shows high accuracy and precision scores compared to other algorithms. A comprehensive study, completed by Dixit and Singh (2023), utilised the Random Forest algorithm to analyse and detect malware using a dataset comprising 138,047 samples. The dataset had a combination of 41,323 benign samples and 96,724 malicious samples.

The model developed by Dixit and Singh (2023) exhibited a high level of accuracy, achieving a rate of 98%, along with a low false positive rate of 3.97%. Nevertheless, certain limitations within the study have been identified, including dataset imbalance characterised by a greater number of malicious samples compared to benign ones. Although this issue could be addressed through the pre-processing phase by narrowing down the samples, failing to do so could significantly impact the model's reliability. In subsequent research conducted by Khammas (2020) and Beaman, Barkworth, Akande, Hakak, Khan (2021), the reliability and efficacy of the Random Forest algorithm in ransomware detection are further substantiated. These studies demonstrated high classification accuracy, with results reaching 95%. For example, the study by Beaman, Barkworth, Akande, Hakak, Khan (2021), utilising the UGRansom2024 dataset, accomplished an accuracy of 96%. Similarly, the research

SAMPLE THESIS

by Khammas (2020) reported a detection precision of 97.74% for malware identification. However, while these results provide strong evidence of Random Forest detection for malware, there still remains a lack of research on its performance in real-time endpoint protection environments.

2.8. Real-Time Endpoint Protection

Real-time endpoint protection is a critical component of modern cybersecurity. It provides continuous monitoring and investigation of systems and immediate responses to any suspicious threats that may arise. Real-time detection is crucial in antivirus software, as it identifies and stops various types of malware, such as ransomware, in real-time. This approach will not only reduce the likelihood of attacks, but also enhance the overall security of systems with rapid mitigation (ReasonLabs, 2023; SentinelOne, 2024). These software systems will work by continuously scanning incoming data for malicious behaviour before it can cause any impact to the system. It will analyse key patterns and indicators such as disk activity, network traffic, and system calls. Since most common attacks are transmitted through data packets or queries, real-time endpoint protection software can perform multiple layers of filtering, decryption, and inspection instantly. Instant response times are crucial in the cybersecurity industry. They enable rapid alerts and instant intervention in the event of any attacks. Aside from blocking suspicious activity, many real-time endpoint protection tools can also try to reverse any malicious changes made during an attack, adding an extra layer of defence. As a result, this boosts user and organisational confidence, ensuring their personal and client data in digital environments are actively monitored and protected.

However, like all software, real-time endpoint protection has its limitations. One notable concern is the vulnerability to insider threats, as individuals with authorised access to systems can bypass protection and acquire sensitive data or implement malware. As most of these actions fall within normal user behaviour within organisations, it would be difficult for real-time endpoint protection software to detect the anomalies (UMATechnology, 2024). More sophisticated security enhancements would need to be implemented to prevent insider threats, and higher access control strategies would need to be put in place.

SAMPLE THESIS

2.9. Gaps in Literature

Although significant progress has been made in research on the application of Machine Learning, particularly with the use of the Random Forest technique for ransomware detection, most of the reviewed studies have concentrated on assessing models in offline or static environments using pre-processed datasets. These studies demonstrate a high accuracy and precision rate; however, they do not explore the effectiveness of the models in real-time endpoint protection, where speed, adaptability, and integration are essential. In addition to this, there is limited research that has assessed the challenges of using these models in dynamic, real-world environments. Due to this, a research gap exists in understanding how the Random Forest algorithm for detection can be adapted to and evaluated within real-time environments.

In light of this, this project will address the gaps by designing and testing a Random Forest-based ransomware detection system using an up-to-date dataset comprising both benign and malicious samples, with an emphasis on simulating real-time endpoint protection. This project aims to evaluate the models' accuracy, precision, and detection speeds, as well as their integration, performance, and real-time protection capabilities. It will bring new insights to the industry by bridging the gap between static Machine Learning evaluations and real-time protection needs.

2.10. Chapter Summary

This chapter examined current research on ransomware and detection methods, emphasising the limitations of traditional techniques such as signature-based and heuristic detection. It also discusses the history and development of ransomware, including its various types, families, and encryption techniques. It also highlights emerging methods such as Machine Learning, particularly the Random Forest algorithm, for improving detection accuracy and identifying zero-day attacks. Moreover, it emphasises the importance of real-time endpoint protection and explores current research employing the Random Forest algorithm, highlighting key gaps in this area.

The next chapter discusses the project management aspect of the ransomware detection project.

SAMPLE THESIS

Chapter 3

Project Management

SAMPLE THESIS

3.1. Chapter Overview

In the previous chapter, ransomware and detection systems were critically analysed through a literature review that provided detailed information on types of Machine Learning, algorithms, existing detection systems, and gaps in current research.

“Project management is the application of processes, methods, skills, knowledge and experience to achieve specific project objectives according to the project acceptance criteria within agreed parameters. Project management has final deliverables that are constrained to a finite timescale and budget.” (Association for Project Management, 2023). For this project, effective project management will be crucial for successful completion without any challenges preventing the project from being finished.

This chapter provides information on the project management of this project, including details on planning, methodology, research, time and resource allocations, and potential risks that could impact the project.

3.2. Project Plan

The project plan comprises several sections that outline the processes and plans in place to ensure the project's completion. This includes the use of the project specification form, Gantt chart, and other management processes.

3.2.1. Project Specification Form

The project specification form was a key element in the planning and management of the project, and it was also used to explain and provide detailed insight into the project to the supervisors. It offered a comprehensive and systematically organised overview of the project, encompassing the title, research question, objectives, and a detailed breakdown of the tasks necessary for completion. This facilitated the strategic planning of essential activities, including the literature review, dataset preparation, model development, real-time simulations, and evaluations, thereby helping to define the project's scope and establish realistic timelines for successful completion. It also facilitated the planning of ethical considerations and compliance, thereby safeguarding the project's integrity. The specification form served as a roadmap, guiding the project from start to finish and facilitating progress monitoring.

SAMPLE THESIS

3.2.2. Gantt Chart

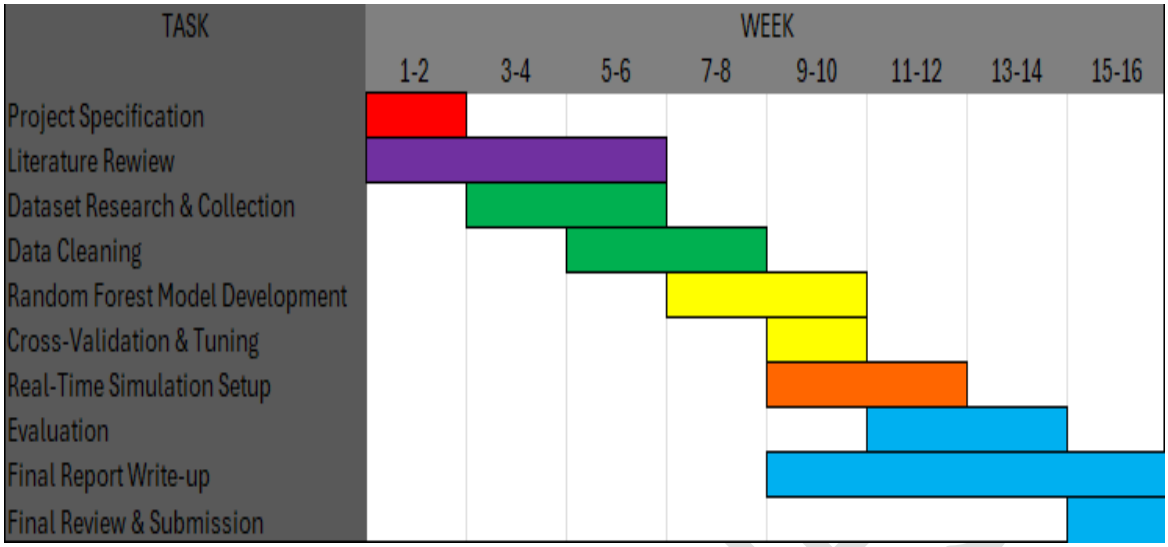

Figure 3.1: Gantt chart of weekly progress and plan

The above Gantt chart was used to plan the project timetable and structure, which has been subdivided into several key tasks over a 16-week duration. The project commences with the completion of the project specification form during weeks 1 and 2, followed by a literature review spanning weeks 1 through 6, which provides insights and foundational knowledge for the research. The dataset research and collection are conducted in weeks 3 to 6, which overlap with the data cleaning in weeks 5 to 8, ensuring that all the data is cleaned and pre-processed for the model training. The Random Forest model development runs from weeks 7 to 10, followed by cross-validation and tuning in weeks 9 to 11, which helps optimise the model's performance. The real-time simulation will be conducted from weeks 11 to 13 to assess the models' practical ability in real-time endpoint protection. This will be followed by the completion and evaluation stage, spanning from weeks 12 to 15, during which the models' precision and reliability will be measured. Simultaneously, the final report will be written from week 10 until the submission date in week 16. This Gantt chart provides a clear and structured roadmap for managing the project effectively, setting deadlines to monitor progress and ensure timely completion of the project.

SAMPLE THESIS

3.3. Monitoring

Throughout the project, it will be supervised by the author and the project supervisor to ensure progress, with deadlines set for each task. These deadlines are established to ensure the project's timely completion and progression.

Additionally, a logbook will be used throughout the entire project to record all activities, meetings, and progress.

3.4. Risk Assessment

Table 3.1. A detailed risk assessment of potential risks that could occur throughout the project, along with the mitigation measures in place to prevent any issues.

Risk No.	Risk Description	Mitigation	Likelihood	Severity
1	Ethical and security considerations	Use publicly available datasets/information.	Low	High
2	Technical or software issues	Create multiple backups and use reliable software.	Low	High
3	Time/deadline issues	Stick to the created Gantt chart and plans to ensure completion on time for the deadline.	Low	Medium
4	Poor model performance	Ensure thorough hyperparameter tuning and data cleaning.	Medium	Low
5	Limited experience with Python	Spend time learning Python libraries and using online resources and tutorials.	Medium	Medium

SAMPLE THESIS

3.5. Chapter Summary

This chapter discussed the overall project management strategy used to ensure the successful delivery of the ransomware detection system. It covered several components needed to ensure smooth project operations, such as the project specification form, which defined the project's objectives and timelines, and the Gantt chart, which mapped out the key tasks across the 15-week timeline. The monitoring of the project was supported through regular meetings with the supervisor and a project logbook to track progress. A risk assessment was also completed to identify any potential issues that could arise, such as ethical considerations, technical issues, or time constraints, along with mitigation strategies in case of any issues.

The following chapter will cover the research design and methodology of the ransomware detection project.

SAMPLE THESIS

Chapter 4

Research Design and Methodology

SAMPLE THESIS

4.1. Chapter Overview

This project will use a Machine Learning methodology to detect ransomware, utilising the Random Forest (RF) algorithm. The primary aim of the project is to create a model capable of differentiating benign from malicious files within a labelled dataset. The project will adhere to an experimental design, wherein the Random Forest algorithm will be trained on a designated ransomware dataset, evaluated through multiple standard performance metrics, and ultimately tested in a simulated real-time environment to assess its responsiveness and accuracy for endpoint protection.

The project will go through several key stages, including data collection and preprocessing, model development, performance evaluation, and real-time simulations. The primary programming language used is Python, which includes popular data libraries such as scikit-learn, pandas, and NumPy. The software used to implement this model will be PyCharm.

4.2. Dataset Description

The dataset being used for this research is the Ransomware Dataset 2024, created by Amjad Hussain. It contains 21,752 samples, evenly split between malicious samples (10,876) and benign samples (10,876). The dataset also includes samples from 26 real-world ransomware families, which makes it well-suited for training a model that recognises multiple threats. It's even sample structure will ensure balanced classification and detection results, supporting a robust performance evaluation. This dataset was selected due to its recency, the variety of ransomware families it contains, and its focus on ransomware. Due to the small sample size of 21,752, the dataset will need to be augmented to increase its size. The aim is to increase the dataset to approximately 80,000 to 90,000 samples by duplicating the current samples and randomising the new samples to generate both ransomware and benign samples.

Table 4.1: Descriptive table showing the dataset before augmentation with all cyber families, types of malware, and total numbers

Type of Malware	Number	Family	Number
Ransomware	332	Cerber	332
Ransomware	335	DarkSide	335
Ransomware	332	Dharma	505
Ransomware	332	Gandcrab	382

SAMPLE THESIS

Ransomware	332	LockBit	438
Ransomware	332	Maze	290
Ransomware	332	Phobos	550
Ransomware	332	Ragnar	133
Ransomware	332	REvil	347
Ransomware	332	Ryuk	449
Ransomware	332	Shade	504
Ransomware	332	WannaCry	497
RAT	332	Gh0st	454
RAT	346	Glupteba	346
RAT	520	NanoCore	520
RAT	506	njRat	506
RAT	512	Remcos	512
RAT	309	Ursnif	309
Trojan	337	Agenttesla	337
Trojan	346	Emotet	346
Trojan	444	Guloader	444
Trojan	322	Qbot	322
Stealer	473	Formbook	473
Stealer	518	Raccoon	518
Stealer	500	RedLine	500
Stealer	527	Snake	527
Benign	10876	Benign	10876
		Total Malware	10876
		Total Benign	10876
		Total	21752

4.3. Data Preprocessing

Before using this dataset for the Machine Learning model, several preprocessing steps are necessary to ensure that the data and all features are clean, well-structured, and in a numerical format, thereby making them suitable for use and training of the Random Forest model.

4.3.1. Data Cleaning

The untouched dataset included several columns that were either specific identifiers or non-numeric, which are irrelevant and incompatible with the Machine Learning model. The removed columns include:

- Cryptographic hashes: MD5 and SHA1
- Description strings: D11Characteristics, magic_number, Subsystem, text_Characteristics, and rdata_Characteristics

SAMPLE THESIS

- Mixed-format columns: Magic, AddressOfEntryPoint, OperatingSystemVersion, ImageVersion, Category, and Family

All of these columns were removed because they contained string data, which is generally incompatible with most standard Machine Learning models. Additionally, they exhibited inconsistent and mixed data types, as well as data that were unrelated to the classification task. Removing these columns will result in a dataset that is clean, well-structured, and comprised of features suitable for numerical modelling, maintaining the prerequisites of the Random Forest Algorithm.

4.3.2. Label Encoding

The sample type column 'class' initially contained string labels, which were 'Benign' and 'Malware'. These labels were transformed into binary numerical values through label encoding, with 'Benign' assigned the value 0 and 'Malware' assigned the value 1. This conversion is essential for the classification task, as the Random Forest algorithm requires numeric variables for effective classification.

4.3.3. Hexadecimal to Integer Conversion

Other features within the dataset also included hexadecimal strings, such as '0x00400000'. These have originated from executable files, such as section sizes and memory locations. This cannot be used for the model, so each string has been converted to its equivalent integer. This action was necessary to maintain the integrity of the data and ensure the model would continue to function correctly.

4.3.4. Category Feature Encoding

Features within the dataset were categorical, such as file_extension, PEType, and MachineType. These had to be converted into numerical formats using one-hot encoding, which transformed the string format into binary variables for each category. This allows the model to interpret categorical data.

4.3.5. Train-Test Splitting

After all the data was pre-processed and cleaned, it had to be divided into two groups: one for testing and one for training. An 80/20 split was used to ensure that both the testing and training sets kept the original class distribution. The 'train_test_split' function from 'sklearn.model_selection' was utilised to prevent any bias in the model's performance on unseen data.

SAMPLE THESIS

4.3.6. Cleaned Dataset

The cleaned and pre-processed dataset was then saved as a new file to ensure safekeeping in case of any complications.

4.4. Model Training/Development

Once the dataset was pre-processed and feature engineering was in place, a specific Random Forest Classifier was used to train the ransomware detection model. The Random Forest algorithm involves creating multiple decision trees and combining their outputs to enhance accuracy.

4.4.1. Model Selection

The model selected for the project was the RandomForestClassifier from the scikit-learn library, with the parameter 35 set for random_state to ensure reproducibility of results. No hyperparameters or tuning were conducted at this stage, as the model was tested using default settings to assess its baseline performance.

4.4.2. Model Training

The model utilised the newly cleaned and pre-processed dataset, which included an 80% training split consisting of 17,401 samples. The .fit() code was used in conjunction with the corresponding labels, features_train and labels_train.

4.4.3. Prediction and Evaluation

After the model was trained, it was tested and evaluated using the testing set, which contained 20% (4,351 samples). The model's performance was assessed using the following metrics:

Accuracy: provides the percentage of correctly classified samples

Precision, Recall, and F1-score: Evaluates both benign and malicious samples to determine the specific performance.

Confusion Matrix: Provides insight into true positives, true negatives, false negatives and false positives to understand the performance of each class.

This was completed using the sklearn.metrics module and imported the accuracy_score, classification_report, confusion_matrix libraries.

SAMPLE THESIS

4.5. 1st Testing Phase

After completing the original dataset and implementing the Random Forest Classifier, the initial testing phase was conducted to verify the functionality of all code and to evaluate different random state seeds for the highest detection rate. The test was run a total of 20 times with different seeds to ensure the best outcome was achieved.

4.6. Dataset Augmentation Methodology

Once the original dataset was completed and working with the Random Forest Classifier, it had to be increased in size to improve the model's overall generalisation capability. This process involved duplicating the original 20,000 samples and applying Gaussian noise to numeric features, which will create slightly modified copies of the original samples.

4.6.1. Selected Columns for Augmentation

All numeric columns were selected for the augmentation phase, whereas non-numeric columns, such as the labels, were excluded, as these must remain unaltered. The “class” column, which contains the labels, was omitted from the augmentation process.

4.6.2. Create Duplicate Copies

The dataset needed to be tripled in size, so three augmented copies were produced. This was a deliberate design choice to increase the overall dataset to over 85,000 samples.

4.6.3. Applying Gaussian Noise

As each new sample was created, Gaussian Noise was applied to differentiate it from the others. The code np.random.normal(loc=1.0, scale=0.05) was utilised to introduce noise with a mean of 1 and a slight standard deviation of 5%.

4.6.4. Merging Augmented Datasets

Once all augmentation was completed, all duplicates and the original dataset were merged into one and then saved as a new CSV file to be used on the Random Forest Classifier.

4.7. 2nd Testing Phase

When the completed augmented dataset is set up and implemented with the Random Forest Classifier, the second testing phase was conducted to verify the functionality of all code and to evaluate different random state seeds for the highest detection rate.

SAMPLE THESIS

The test was run a total of 20 times with different seeds to ensure the best outcome was achieved.

4.8. Real-Time Detection

To simulate real-time ransomware detection, numerous unseen samples from the dataset were processed through the trained Random Forest model. While this approach does not involve live network traffic or file monitoring, it offers a simulation of how the endpoint protection system would classify files upon assessment.

4.8.1. Real-Time Detection Code

Firstly, the samples from the dataset will be selected using sample_from_dataset, which will represent the files that will arrive on the system when scanned in real time. Following this, the features (sample_data) and the true labels (sample_labels) are retrieved using .iloc. Once this is completed, the data is then employed with the trained Random Forest Classifier to predict the class of each selected sample.

4.9. 3rd Testing Phase

The last completed test involves evaluating various sample sets to verify the accuracy of the Random Forest Classifier's predictions across all samples.

4.10. Software Used and Python Libraries

- Software - PyCharm 2024.3.5
- Libraries – Pandas, sklearn.model_selection, sklearn.ensemble, sklearn.metrics, NumPy

4.11. Chapter Summary

This chapter explained the research design and methodology of the ransomware detection project. It outlined key stages, including the dataset selection and description, preprocessing, model training, testing, and real-time simulation. The Ransomware Dataset 2024 was used, cleaned, and then augmented to 87,006 samples using Gaussian noise. The data preprocessing steps involved cleaning the dataset, encoding, and splitting it into training and testing sets. The Machine Learning model was built and trained using scikit-learn in Python, then tested in multiple phases

SAMPLE THESIS

to measure accuracy. Lastly, the real-time detection was simulated to evaluate the model's practical application, and all work was completed using PyCharm and Python libraries.

The next chapter will discuss the overall design of the ransomware detection model, including various screenshots, and provide in-depth details of each component.

SAMPLE THESIS

Chapter 5

Design

SAMPLE THESIS

5.1. Chapter Overview

The previous chapter provided a detailed insight into the project's methodology and what would be required if it were completed under another individual.

The purpose of a research design is to serve as a framework or blueprint that guides the collection, analysis, and interpretation of data in a study. It will serve as a plan to guide the project's direction and ensure that the methods used align with the set objectives. The design chapter is a vital step as it assists in completing the project and ensures that each objective is achieved without any issues. The design chapter will provide a detailed discussion of the project's design, accompanied by diagrams and screenshots to ensure a clear understanding.

5.2. Project Design Methodology

Having an appropriate design methodology will be essential to ensure the smooth completion of the project objectives. When selecting a design methodology for this project, it was ideal to use a methodology with clear transitions to each step; for this reason, the waterfall methodology approach was selected. The waterfall methodology is a well-structured and sequential approach that perfectly matches the nature of this project. At each stage, data preprocessing to model evaluation was completed before moving on to the next, enabling steady progress and consistent documentation of each step. The waterfall methodology is still a well-used and relevant approach in research projects as it's “super organised. It breaks the project down into clear steps, including gathering requirements, designing, developing, testing, and deploying. This way, individuals can focus on one thing at a time, ensuring everything is done well” (Team EMB, 2024).

5.3. Overall Project Design

This section of the design chapter will discuss the overall project plan, providing insight into each stage required to complete the project. It will provide information on the four main components of the project and why they are essential for the overall completion.

SAMPLE THESIS

5.3.1. Waterfall Diagram of Project

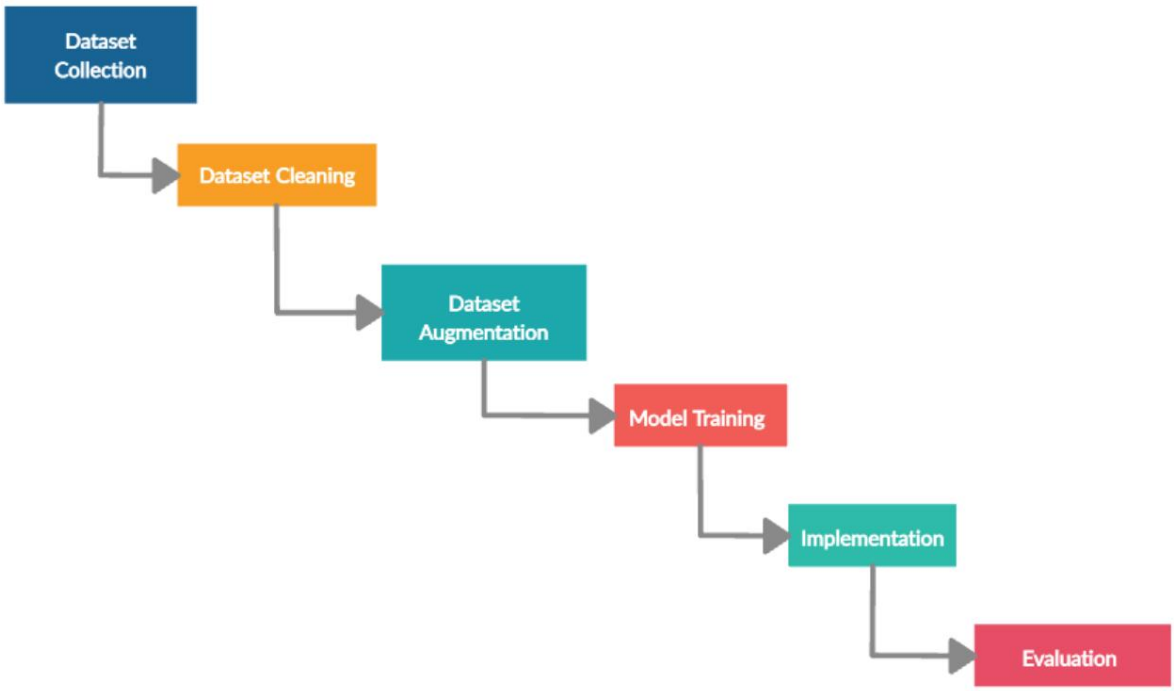

Figure 5.1: Waterfall diagram showing each main component of the project. Created by Kai Johnson using Creately.

5.3.2. Overall Design Discussion

This project will consist of six central components and will follow the waterfall methodology, advancing through each critical stage in sequence. The first stage of the design is Dataset Collection, which involves obtaining a dataset with benign and malicious files to serve as the foundation for training and testing the model. After collection, the Dataset Cleaning was completed to prepare this data for the Machine Learning model. This included several actions, such as removing irrelevant columns like file hashes, converting hexadecimal values to integers, and changing class categories to numeric values. Once the dataset was cleaned, it underwent data augmentation. Due to the small sample size of 21,752 used for the Machine Learning model, it needed to be increased to approximately 80,000 to 90,000 samples. To achieve this, the original dataset was expanded by applying Gaussian noise to numerical features, generating new and realistic features and variations of the original dataset entries, and increasing the dataset's capacity to enhance generalisation. After the dataset had been cleaned and augmented, Model Training was completed using the Random Forest Classifier. This allows the model to learn the distinguishing

patterns between the ransomware and benign files from the input features. Following the model training, the Implementation phase will involve applying the model in a tested or simulated environment to assess its practical ability in real-time protection. Lastly, its performance will be assessed using accuracy, precision, recall, F1-score, and a confusion matrix to demonstrate its effectiveness in detecting ransomware threats.

5.4. Dataset Design

This section displays each part of the dataset design, including its cleaning stages, with diagrams and screenshots of the process.

5.4.1. Dataset Cleaning Diagram

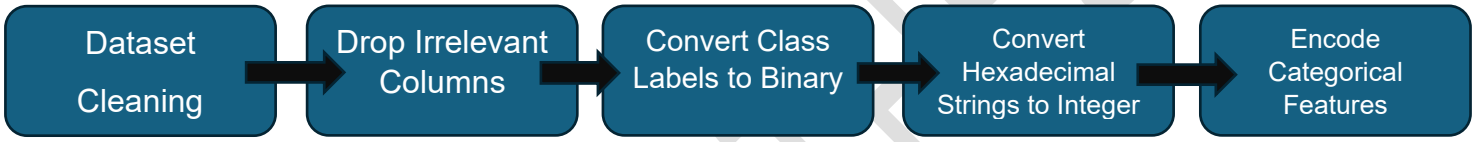

Figure 5.2: Diagram showing each component of the dataset cleaning stage. Created by Kai Johnson using Microsoft Word.

5.4.2. Dataset Cleaning Process

Dataset cleaning is an essential step for a Machine Learning approach, as it involves identifying and removing irrelevant, missing, and duplicate data. The primary goal of cleaning is to ensure that all data is accurate, error-free, and consistent. Because raw data can be noisy, inconsistent, and incomplete, failing to clean it could reduce the model's precision and accuracy, as well as the reliability of the results it produces (GeeksforGeeks, 2018).

```
import pandas as pd
from sklearn.model_selection import train_test_split

ransom_dataset = pd.read_csv("Final_Dataset_without_duplicate_main.csv")

ransom_dataset.drop(columns=["md5", "sha1", "magic_number", "Subsystem", "DllCharacteristics",
    "text_Characteristics", "rdata_Characteristics", "Magic", "AddressOfEntryPoint",
                "OperatingSystemVersion", "ImageVersion", "Category", "Family"], inplace=True)
```

Figure 5.3: Code snippet used to import libraries, read the dataset for cleaning and columns being dropped.

SAMPLE THESIS

Regarding Figure 5.3, the primary library used to clean the dataset is pandas, which will be imported using Python. This library provides functions to clean, analyse, explore, and manipulate data, which are essential for the dataset used in this project. The code pd.read.csv is employed to read the dataset, which will then be cleaned. The initial step in cleaning involves dropping any irrelevant or non-numeric data that would not be useful in Machine Learning training. This includes data such as hashes like MD5 or SHA1.

```
ransom_dataset["Class"] = ransom_dataset["Class"].map({"Benign": 0, "Malware": 1})
```

Figure 5.4: Code snippet of class labels being changed to binary format.

Following Figure 5.3, Figure 5.4 shows the code that will convert the label column in the dataset, indicating which samples are malicious and benign. This is vital because Machine Learning will utilise a binary format, so all labels must be changed to the corresponding values, with Benign as 0 and Malware as 1.

```
def convert_hex_to_int(value):
    try:
        return int(value, 16)
    except:
        return None


hexadecimal_columns = [
    'EntryPoint', 'bytes_on_last_page', 'pages_in_file', 'relocations',
    'size_of_header', 'min_extra_paragraphs', 'max_extra_paragraphs',
    'init_ss_value', 'init_sp_value', 'init_ip_value', 'init_cs_value',
    'over_lay_number', 'oem_identifier', 'address_of_ne_header',
    'SizeOfCode', 'SizeOfInitializedData', 'SizeOfUninitializedData',
    'BaseOfCode', 'BaseOfData', 'ImageBase', 'SectionAlignment',
    'FileAlignment', 'SizeOfImage', 'SizeOfHeaders', 'Checksum',
    'SizeofStackReserve', 'SizeofStackCommit', 'SizeofHeapCommit',
    'SizeofHeapReserve', 'LoaderFlags',
    'text_VirtualSize', 'text_VirtualAddress', 'text_SizeOfRawData',
    'text_PointerToRawData', 'text_PointerToRelocations',
    'text_PointerToLineNumbers',
    'rdata_VirtualSize', 'rdata_VirtualAddress', 'rdata_SizeOfRawData',
    'rdata_PointerToRawData', 'rdata_PointerToRelocations',
    'rdata_PointerToLineNumbers'
]


for columns in hexadecimal_columns:
    ransom_dataset[columns] = ransom_dataset[columns].apply(convert_hex_to_int)
```

Figure 5.5: Code snippet of converting hexadecimal strings to integers.

In reference to Figure 5.5, the next step in data cleaning is converting the hexadecimal strings to integers so the Machine Learning model can understand the data. This first step involves defining the function to convert hexadecimal strings to base-10 integers,

which transforms it into a more suitable numeric format for the Machine Learning model to process. Next, the columns identified as containing hexadecimal values need to be specified; the code in Figure 5.5 shows each column that was found and required conversion. The final snippet of code applies the conversion to all identified columns, which will transform the hexadecimal strings to integers, making it usable for the Machine Learning model.

```
ransom_dataset = pd.get_dummies(ransom_dataset, columns=["file_extension", "PEType", "MachineType"], drop_first=True)
```

Figure 5.6: Code snippet of hot encoding categorical features.

The last part of the data cleaning process was to one-hot encode the categorical feature, and some of them contained string labels not suitable for Machine Learning. This code creates new binary columns for each category, allowing for the model to interpret these values.

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	md5	sha1	file_extension	EntryPoint	PEType	MachineType	magic_number	bytes_on_last_page	pages_in_file	relocations	size_of_header	min_extra_paragraphs	max_extra_paragraphs
2	d5aa2b2506dd71b65307bb551a6a099d	62f4d55a1bb8396e493d7815dc44febed7161535	exe	0x108ec	PE32+	AMD AMD64	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
3	8d1178cec97f96a9ede3efe2f30ef30e	ca40945a4e35d48794a5e13aacf16b8ae1caddeb	exe	0x25cf9c	PE32+	AMD AMD64	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
4	c08d7afb7966f452cf5aa9283fe8880a	6f47b50480bea61ae4dfb75c12582990cc5ce770	exe	0x2a7e	PE32	Intel 386 or later, and compatibles	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
5	3554a3a99e7ffb57552aedcb2520f1d2	8c3d5b506a8d55f42f4c83ead3e4341255d1acfc	exe	0x64ca0	PE32+	AMD AMD64	MZ	0x0090	0x0003	0x0004	0x0000	0x0000	0xFFFF
6	5be9511d42136df49d9501680e84109f	0a7d0c68c591db2f197b2169d226bebe6e9bdabe	exe	0x1b9ae	PE32	Intel 386 or later, and compatibles	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
7	d537acb8f56a1ce206bc35cf8ff959c0	0bb491f62b77df737801b9ab0fd14fa12d43d254	exe	0x0208	PE32	Intel 386 or later, and compatibles	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
8	8a40b88f514ef21f74052d22e2d98750	524e636f22bb8a5c2f7947727d5f91dd3dd05972	exe	0x36000	PE32	Intel 386 or later, and compatibles	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
9	492c895eef1bbf512d2a62953c6dc490	b10ffab94d90d85bfb94acb86cfce7368d0df3e3	exe	0x9fb4	PE32	Intel 386 or later, and compatibles	MZ	0x0080	0x0001	0x0000	0x0004	0x0000	0xFFFF
10	b36ae3f1cbd1e4d8df6d5c5e7268bc91	4ed46940b19333be5a8590dc369a2bdb501382ff	exe	0x57c2	PE32	Intel 386 or later, and compatibles	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
11	b8e27c15d3f05a66ab6dd2db000ae059	6667bcec444718db2568d6642f8a495c05d95ae7	exe	0x1242	PE32	Intel 386 or later, and compatibles	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
12	7f01c29f7833f03c556620f868b0eb13	1f764df0a3913b655b38a8c76fe416b367f395ee	exe	0x419ad0	PE32+	AMD AMD64	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
13	5e53b40cf972f4eb08990999ce17c5c8	ca5d95f3be661dbcabe52d477ad78f19b832c532	exe	0x2800a	PE32	Intel 386 or later, and compatibles	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
14	743bea93cf7e254a0a66867cb64f2e1d	8d8d2e104fbb846836d825cddc1180bc58a66db4	exe	0x5c9e	PE32	Intel 386 or later, and compatibles	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
15	db427ed458e5da02ef41535256f12504	0a2f152cd2bc454039e84a95539ad8fcd883748a	exe	0xd6b9	PE32	Intel 386 or later, and compatibles	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
16	639fe0ae9b405d7d0edc999af26b9ae1	88a141f79709d69fc5c69e1be9f624d0e1ca0db6	exe	0x1fbc30	PE32+	AMD AMD64	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
17	f2d5e887bb4e3e88684c2a4039f368d5	b76a264b7aa5c98173c788c32e88654fb25af718	exe	0x12cb	PE32	Intel 386 or later, and compatibles	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
18	438862a077e46a0462c776b6d2c1bbe5	04e54eb9b199f94e4ad1c10ce1de84825caedf7c	exe	0x4261be	PE32	Intel 386 or later, and compatibles	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
19	57a327f0fb9c17741224468cb1f7c657	fb84608abdda98ca8a33f68c53ae48eff1af1ced	exe	0x14c0	PE32+	AMD AMD64	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
20	792bdde643aaefa20f8138dc94fa21c4	a0c9b91e9a1d269f371ef41e1308e1de81b10d0a	exe	0x1cc2	PE32+	AMD AMD64	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
21	72a02361ea6a72ed57247047b780df19	3bdc295eae546ba86fbd5a98ee78026fab0340b8	exe	0x1cc2	PE32+	AMD AMD64	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
22	429cb2ec4192908991ec50f18ea62bf0	256aee2fc563563808d81d2d33b4a68715f3e034	exe	0x19f8	PE32	Intel 386 or later, and compatibles	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
23	bbdd5467c224c31f3a25e199949b082c	77f2cebeaa34355b0b041c197e3ae59ed351fe0b	exe	0x613d4	PE32+	AMD AMD64	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
24	69547142db931e489d088fab51f42821	15d88278758159d29c09d465d1083f62252419fc	exe	0x9b10	PE32+	AMD AMD64	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
25	96c1c6cba52837f8fa37ae285a5f8d2e	db1eca343c975a51891b34bd6ca4400967d8f114	exe	0x313e	PE32	Intel 386 or later, and compatibles	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
26	9cd2b1e25addbf1c3980a7ee5e678d67	632240b967dee7a588637795b0d89d73329b5598	exe	0x1260	PE32+	AMD AMD64	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
27	bc0acbcdfad7ef304da34874aba54100	cc863fd27f85214217a3d6199cfbc4bc75526699	exe	0xdd8c	PE32+	AMD AMD64	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
28	e383e24470289faead6fbd929e988d05	cf07f90250fc3ba41002cc265fee8e51777faba3	exe	0x2e5e	PE32	Intel 386 or later, and compatibles	MZ	0x0040	0x0001	0x0000	0x0002	0x0000	0xFFFF
29	a116a71625123b74d7fb1e648b4b5e3f	2dbdd7e87fc53bc6683bd8d1caec4a7eb638ef1d	exe	0x3760	PE32	Intel 386 or later, and compatibles	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
30	dfeb45706955e4486d464b0bd62c2a0e	05b5dcd6d701da964f5443c8a33062860af29af9	exe	0x56ba10	PE32+	AMD AMD64	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
31	098062dde5741b0b42e73060a1b95db0	803e9fd3f740cfebb06333a7e056e6b6dbdc10d1	exe	0x3248	PE32	Intel 386 or later, and compatibles	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
32	c87556b91fe1e07318fe4a99c8247903	21ea38ea00ce1f5385edfeb1001b6167474f8b65	exe	0x3dfe	PE32	Intel 386 or later, and compatibles	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
33	0e2cdc4f20b34d298c46f46f6baa1270	f8dc1e52a8d7de993dbcbc55ce2650a55f3ea971	exe	0x5c9e	PE32	Intel 386 or later, and compatibles	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
34	70f5712af69e43e691a6adef7f9716fb	326dbcbd77dbe38fd938684f2a506b9bf8dcdfd7	exe	0x14e0	PE32	Intel 386 or later, and compatibles	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
35	abcb6734f854bf3a4aca7d236c2069d3	0af6b1602f5d70d51a56320d8847eb73b62cda99	exe	0x514417	PE32+	AMD AMD64	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
36	1b3163e0cd8ba2c646dae5c55685086d	ea7f5ed2032804ba2e1c8b338000a09347325305	exe	0x23e16	PE32	Intel 386 or later, and compatibles	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF
37	e651d546c79039e9fe3e59e49f7a36c8	24753933dea08149b00da99940814412c7db293d	exe	0x333a	PE32	Intel 386 or later, and compatibles	MZ	0x0090	0x0003	0x0000	0x0004	0x0000	0xFFFF

Final_Dataset_without_duplicate

Figure 5.7: Original dataset before cleaning.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
1	EntryPoint	bytes_on_last_page	pages_in_file	relocations	size_of_header	min_extra_paragraphs	max_extra_paragraphs	init_ss_value	init_sp_value	init_ip_value	init_cs_value	over_lay_number	oem_identifier	address_of_ne_header	SizeOfCode	SizeOfInitializedData	SizeOfUninitializedData
2	67820	144	3	0	4	0	65535	0	184	0	0	0	0	248	70144	53760	0
3	2477980	144	3	0	4	0	65535	0	184	0	0	0	0	280	3527168	1647616	0
4	10878	144	3	0	4	0	65535	0	184	0	0	0	0	128	3072	2048	0
5	412832	144	3	4	0	0	65535	0	139	0	0	0	0	128	2240000	330240	0
6	113070	144	3	0	4	0	65535	0	184	0	0	0	0	128	104960	2560	0
7	520	144	3	0	4	0	65535	0	184	0	0	0	0	176	512	6144	0
8	221184	144	3	0	4	0	65535	0	184	0	0	0	0	128	165888	44544	0
9	40884	128	1	0	4	0	65535	0	184	0	0	0	0	112	71680	17920	9216
10	22466	144	3	0	4	0	65535	0	184	0	0	0	0	256	67072	39936	0
11	4674	144	3	0	4	0	65535	0	184	0	0	0	0	280	42496	597504	0
12	4299472	144	3	0	4	0	65535	0	184	0	0	0	0	128	1175552	69632	3121152
13	163850	144	3	0	4	0	65535	0	184	0	0	0	0	272	581632	284672	0
14	23710	144	3	0	4	0	65535	0	184	0	0	0	0	128	15872	1536	0
15	54969	144	3	0	4	0	65535	0	184	0	0	0	0	232	374784	250368	0
16	2079792	144	3	0	4	0	65535	0	184	0	0	0	0	296	2136064	1523712	0
17	4811	144	3	0	4	0	65535	0	184	0	0	0	0	272	16384	15872	0
18	4350398	144	3	0	4	0	65535	0	184	0	0	0	0	128	4342272	147456	0
19	5312	144	3	0	4	0	65535	0	184	0	0	0	0	128	6656	286208	512
20	7362	144	3	0	4	0	65535	0	184	0	0	0	0	128	4096	2043904	0
21	7362	144	3	0	4	0	65535	0	184	0	0	0	0	128	4096	2043904	0
22	6648	144	3	0	4	0	65535	0	184	0	0	0	0	192	53248	8192	0
23	398292	144	3	0	4	0	65535	0	184	0	0	0	0	272	569344	195584	0
24	39696	144	3	0	4	0	65535	0	184	0	0	0	0	248	162816	97280	0
25	12606	144	3	0	4	0	65535	0	184	0	0	0	0	128	4608	2048	0
26	4704	144	3	0	4	0	65535	0	184	0	0	0	0	272	40448	613888	0
27	56716	144	3	0	4	0	65535	0	184	0	0	0	0	280	59392	28160	0
28	11870	64	1	0	2	0	65535	0	184	0	0	0	46080	64	4096	1536	0
29	14176	144	3	0	4	0	65535	0	184	0	0	0	0	216	159744	1265664	0
30	5683728	144	3	0	4	0	65535	0	184	0	0	0	0	352	7279616	2974720	0
31	12872	144	3	0	4	0	65535	0	184	0	0	0	0	232	31232	22528	0
32	15870	144	3	0	4	0	65535	0	184	0	0	0	0	128	8192	2048	0
33	23710	144	3	0	4	0	65535	0	184	0	0	0	0	128	15872	13312	0
34	5344	144	3	0	4	0	65535	0	184	0	0	0	0	128	84480	115712	1536
35	5325847	144	3	0	4	0	65535	0	184	0	0	0	0	128	904704	312320	0
36	146966	144	3	0	4	0	65535	0	184	0	0	0	0	256	152576	508928	0
37	13114	144	3	0	4	0	65535	0	184	0	0	0	0	128	5120	31744	0

cleaned_ransomware_dataset

Figure 5.8: New dataset after cleaning.

Once the data cleaning is complete, the dataset is saved and reviewed to show the conversion of all columns and the dropped columns. In reference to Figures 5.6 and 5.7, you can see the change from before cleaning to after, as well as how the columns have been transformed.

5.5. Dataset Augmentation Design

As the current dataset used for the ML model only has 21,752 samples, it was discussed that a larger dataset would be more beneficial for the Random Forest algorithm. The ideal dataset size would be 80,000 to 90,000, and to complete this task, it will be designed using NumPy. NumPy is a highly used Python library that can be used to apply transformations to datasets. To enhance the model's generalisability and robustness, the dataset will be augmented to 87,000 samples, which will ensure the model will be trained on a more varied dataset and reduce the risks of overfitting.

```
numeric_cols_for_augmentation = ransom_dataset.select_dtypes(include=[np.number]).columns.tolist()
numeric_cols_for_augmentation.remove("Class")

number_of_new_copies = 3

augmented_data = []
```

Figure 5.9: Code snippet for selecting data for data augmentation.

SAMPLE THESIS

The first part of the data augmentation process was to select all the data that would be augmented and the data that would not be changed. Concerning Figure 5.9, all numeric data were selected in the dataset to be augmented. Following this, the class column must not be changed to ensure that the labels are not altered, as these are the labels to differentiate benign from malicious samples. After the data has been selected, the total number of copies that will be made is 3, which takes the total number of samples from 21,752 to 87,008. This makes a large number perfect for the model.

```
for _ in range(number_of_new_copies):

    copy_ransom_dataset = ransom_dataset.copy()

    for column_names in numeric_cols_for_augmentation:
        noise = np.random.normal(loc=1.0, scale=0.05, size=len(copy_ransom_dataset))
        copy_ransom_dataset[column_names] = copy_ransom_dataset[column_names] * noise
        copy_ransom_dataset[column_names] = copy_ransom_dataset[column_names].round().astype(int)

    augmented_data.append(copy_ransom_dataset)
```

Figure 5.10: Code snippet for altering data for data augmentation.

Following Figure 5.9, the next step of the data augmentation design is to duplicate all the data and apply noise to each. The dataset will be copied a total of 3 times, tripling the size of the samples for the model. After this, each numeric feature will be slightly altered by multiplying it by the random Gaussian noise, which represents a +5% variation. These changes will alter all new samples and simulate real-world fluctuations, introducing variability and maintaining the structure of the sample data.

```
df_augmented = pd.concat([ransom_dataset] + augmented_data, ignore_index=True)

print("Dataset Augmentation Complete")
print("Original size of Dataset:", len(ransom_dataset))
print("New augmented size:", len(df_augmented))
```

Figure 5.11: Code snippet for merging newly created samples and the original dataset.

The final step in the design of data augmentation is to add all newly generated samples and merge them with the original dataset. Combining these will create one

comprehensive dataset for the Machine Learning model using the Random Forest Classifier.

```
Dataset Augmentation Complete
Original size of Dataset: 21752
New augmented size: 87008
```

Figure 5.12: image showing the completion of augmentation with the old and new sizes of the dataset.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	AA	AB	AC
21727	641626	144	3	0	4	0	65535	0	184	0	0	0	0	128	633856	2048	0	8192	647168	4194304	8192	512	663552	512	0	1048576	4096	4096	1048576
21728	703970	144	3	0	4	0	65535	0	184	0	0	0	0	128	696320	2048	0	8192	704512	4194304	8192	512	720896	512	0	1048576	4096	4096	1048576
21729	593370	144	3	0	4	0	65535	0	184	0	0	0	0	128	585216	9216	0	8192	598016	4194304	8192	512	622592	512	0	1048576	4096	4096	1048576
21730	1501318	144	3	0	4	0	65535	0	184	0	0	0	0	128	1493504	2048	0	8192	1507328	4194304	8192	512	1523712	512	0	1048576	4096	4096	1048576
21731	626854	144	3	0	4	0	65535	0	184	0	0	0	0	128	619008	2048	0	8192	630784	4194304	8192	512	647168	512	0	1048576	4096	4096	1048576
21732	846286	144	3	0	4	0	65535	0	184	0	0	0	0	128	838144	1536	0	8192	851968	4194304	8192	512	868352	512	0	1048576	4096	4096	1048576
21733	846286	144	3	0	4	0	65535	0	184	0	0	0	0	128	8540996	34564	0	33170	8607108	68764420	33170	1298	8722792	1298	0	17073526	16534	16534	17073526
21734	606774	144	3	0	4	0	65535	0	184	0	0	0	0	128	8540996	34564	0	33170	8607108	68764420	33170	1298	8722792	1298	0	17073526	16534	16534	17073526
21735	499406	144	3	0	4	0	65535	0	184	0	0	0	0	128	8540996	34564	0	33170	8607108	68764420	33170	1298	8722792	1298	0	17073526	16534	16534	17073526
21736	806254	144	3	0	4	0	65535	0	184	0	0	0	0	128	798208	33792	0	8192	811008	4194304	8192	512	860160	512	0	1048576	4096	4096	1048576
21737	0	144	3	0	4	0	65535	0	184	0	0	0	0	128	32256	3584	0	8192	811008	4194304	8192	512	49152	512	0	4194304	16384	8192	1048576
21738	709482	144	3	0	4	0	65535	0	184	0	0	0	0	128	701440	2048	0	8192	712704	4194304	8192	512	729088	512	0	1048576	4096	4096	1048576
21739	518498	144	3	0	4	0	65535	0	184	0	0	0	0	128	510464	5120	0	8192	524288	4194304	8192	512	540672	512	0	1048576	4096	4096	1048576
21740	1478218	144	3	0	4	0	65535	0	184	0	0	0	0	128	1470464	2048	0	8192	1482752	4194304	8192	512	1499136	512	0	1048576	4096	4096	1048576
21741	565826	144	3	0	4	0	65535	0	184	0	0	0	0	128	558080	2048	0	8192	573440	4194304	8192	512	589824	512	0	1048576	4096	4096	1048576
21742	565826	144	3	0	4	0	65535	0	184	0	0	0	0	128	9474834	96544	0	33170	9532676	68764420	33170	1298	9765490	1298	0	17073526	16534	16534	17073526
21743	543586	144	3	0	4	0	65535	0	184	0	0	0	0	128	535552	5632	0	8192	548864	4194304	8192	512	565248	512	0	1048576	4096	4096	1048576
21744	543586	144	3	0	4	0	65535	0	184	0	0	0	0	128	157220	1316900	8264	16534	206696	68764420	16534	1298	2556726	4132	0	17073526	16534	16534	17073526
21745	613054	144	3	0	4	0	65535	0	184	0	0	0	0	128	605184	2048	0	8192	614400	4194304	8192	512	630784	512	0	1048576	4096	4096	1048576
21746	613054	144	3	0	4	0	65535	0	184	0	0	0	0	128	4785302	33170	0	33170	4822802	68764420	33170	1298	5333142	1298	0	17073526	16534	16534	17073526
21747	13888	144	3	0	4	0	65535	0	184	0	0	0	0	216	26624	141824	2048	4096	32768	4194304	4096	512	245760	1024	0	1048576	4096	4096	1048576
21748	559542	144	3	0	4	0	65535	0	184	0	0	0	0	128	551424	2048	0	8192	565248	4194304	8192	512	581632	512	0	1048576	4096	4096	1048576
21749	559542	144	3	0	4	0	65535	0	184	0	0	0	0	128	6453010	37398	0	33170	6523254	68764420	33170	4132	6640480	4132	0	17073526	16534	16534	17073526
21750	631538	144	3	0	4	0	65535	0	184	0	0	0	0	128	623616	2048	0	8192	638976	4194304	8192	512	655360	512	0	1048576	4096	4096	1048576
21751	631538	144	3	0	4	0	65535	0	184	0	0	0	0	128	5522006	71542	0	33170	5599318	68764420	33170	1298	5772850	1298	0	17073526	16534	16534	17073526
21752	631538	144	3	0	4	0	65535	0	184	0	0	0	0	128	6894152	22066	0	33170	7357714	68764420	33170	1298	7473302	1298	0	17073526	16534	16534	17073526
21753	660398	144	3	0	4	0	65535	0	184	0	0	0	0	128	6894152	22066	0	33170	7357714	68764420	33170	1298	7473302	1298	0	17073526	16534	16534	17073526
21754																													
21755																													
21756																													
21757																													
21758																													
21759																													
21760																													
21761																													
21762																													

Figure 5.13: Before dataset augmentation

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	AA	AB	AC
86989	842305	146	3	0	4	0	66522	0	185	0	0	0	0	132	8478668	36416	0	36378	8368275	70537982	32875	1457	8089083	1302	0	17225940	16396	16986	17489340
86990	611058	138	3	0	4	0	70416	0	183	0	0	0	0	133	8621502	34401	0	31026	8963510	69896399	33236	1272	7843418	1412	0	17194618	16293	17342	18163283
86991	498121	148	3	0	4	0	67105	0	182	0	0	0	0	121	8701445	36587	0	32852	8789233	70109852	33904	1306	8496557	1264	0	17584227	16182	17092	17628336
86992	802070	138	3	0	4	0	66001	0	184	0	0	0	0	132	876455	33084	0	8047	871898	4380233	8789	536	876163	448	0	1087579	4048	4228	997531
86993	0	155	3	0	4	0	63256	0	179	0	0	0	0	127	32324	3726	0	8924	840858	4119274	7773	500	47629	518	0	4064077	16811	8306	1052754
86994	725870	140	3	0	4	0	69247	0	180	0	0	0	0	122	735665	2233	0	7958	657855	4057216	8627	582	736755	520	0	1056302	4189	3964	976714
86995	496028	150	3	0	4	0	70472	0	178	0	0	0	0	129	534794	5737	0	7828	523317	4105827	8102	502	493503	506	0	1113630	4202	4027	1223935
86996	1497464	140	3	0	4	0	64922	0	186	0	0	0	0	141	1496688	2007	0	8369	1538358	3720463	7992	464	1458232	464	0	1164963	4451	4368	1088674
86997	629963	153	3	0	4	0	64899	0	197	0	0	0	0	126	632006	2020	0	8511	613540	4176133	7576	477	573309	515	0	993245	3951	4173	1117840
86998	550571	143	3	0	4	0	68212	0	186	0	0	0	0	129	9408692	94202	0	34159	9930676	67452504	36287	1358	9069222	1213	0	16863614	16866	17864	16131095
86999	564149	135	3	0	4	0	59720	0	175	0	0	0	0	134	524989	5377	0	7837	565431	4180444	8529	490	582216	478	0	1111104	4125	4171	1012381
87000	570469	149	3	0	4	0	63791	0	190	0	0	0	0	122	158670	1347312	7884	16539	210048	71624731	16805	1294	2714947	4011	0	16855441	17695	16362	15893088
87001	665589	140	3	0	4	0	67619	0	186	0	0	0	0	118	598071	1980	0	8311	617685	4462802	7726	555	636047	538	0	1050509	4021	4350	1003029
87002	653943	143	3	0	4	0	64816	0	182	0	0	0	0	129	4619813	35065	0	32312	4644458	71465608	33466	1346	5505114	1160	0	16421728	17411	17793	17470485
87003	12916	140	3	0	4	0	64788	0	189	0	0	0	0	211	27942	142699	1942	3810	35376	4006668	4016	555	239409	1067	0	1040462	4253	4193	1132492
87004	569645	142	3	0	4	0	59407	0	204	0	0	0	0	125	555770	2046	0	8547	536473	4390409	8727	557	568053	479	0	1062384	4073	4097	1079836
87005	610612	135	3	0	4	0	65389	0	182	0	0	0	0	133	6213448	36216	0	37164	6157336	71785753	34669	4066	6689768	4221	0	17908965	17494	16635	17587434
87006	632720	145	3	0	4	0	65409	0	184	0	0	0	0	124	658020	2114	0	8137	684882	4177972	7890	516	652888	488	0	1100799	4315	4133	1013303
87007	626893	135	3	0	4	0	66846	0	165	0	0	0	0	125	5272309	71727	0	32453	5421876	65421145	33831	1344	5632302	1330	0	16223643	17646	16171	16023262
87008	647032	150	3	0	4	0	69042	0	191	0	0	0	0	122	6296524	24626	0	33100	7232006	67723616	34312	1381	7643760	1336	0	18548697	16272	17390	17171479
87009	693609	142	3	0	4	0	62670	0	176	0	0	0	0	121	6687171	21421	0	31378	7039346	70334668	33060	1287	7685764	1276	0	17434942	17943	17336	18596543
87010																													
87011																													
87012																													
87013																													
87014																													
87015																													
87016																													
87017																													

Figure 5.14: after dataset augmentation.

5.6. Random Forest Classification

The last part of the design phase includes the implementation of the Random Forest algorithm. For this, I used the Random Forest Classifier, which is a standard supervised Machine Learning technique. The first part of the RF Classifier design starts with splitting the data using train_test_split from the sklearn.model_selection library, where the dataset is divided into 80% and 20%. The 80% is used for training, and the remaining 20% for testing. The stratify parameter was also used to ensure the balance of benign and malicious samples in each of the train and testing splits. The Classifier was also trained with .fit() on the training data, and the test data will be evaluated using specific metrics such as accuracy, precision, recall, and the confusion matrix, all calculated using sklearn.metrics. The reasons for using the RF algorithm from this library are its efficiency and ability to handle large amounts of data effectively, as well as its method of constructing multiple decision trees and combining their outputs to improve results.

```
features = ransom_dataset.drop( labels: "Class", axis=1)
labels = ransom_dataset["Class"]
```

Figure 5.15: Code snippet for label and feature separation.

In reference to Figure 5.15, the code for separating features and labels is displayed. This will divide the dataset used for the model into features and labels. The class column, which contains the target classification (0 for benign and 1 for malware), is removed from the feature set to prevent any data leakage.

```
features_train, features_test, labels_train, labels_test = train_test_split(
    *arrays: features, labels, test_size=0.2, random_state=42, stratify=labels
)

print("Train-test complete")
print(f"Training set: {features_train.shape}, Testing set: {features_test.shape}")
```

Figure 5.16: Code snippet for train and test splitting.

Secondly, the dataset is split into 80% for training and 20% for testing using the train_test_split. The test_size represents the 20% following the random state seed, which ensures the same split every time the test is run. Lastly, the stratify=labels parameter will ensure consistency in both sets, avoiding any imbalance.

```
model = RandomForestClassifier(random_state=42)
model.fit(features_train, labels_train)


test_pred = model.predict(features_test)


print("\n=== Evaluation ===")
print("Accuracy:", accuracy_score(labels_test, test_pred))
print("\nClassification Report:\n", classification_report(labels_test, test_pred))
print("\nConfusion Matrix:\n", confusion_matrix(labels_test, test_pred))
```

Figure 5.17: code snippet for Model training and evaluations.

Finally, the Random Forest Classifier is employed and trained on the training datasets. The Random Forest method works by constructing multiple decision trees, which combine their outputs for precise predictions. These predictions are then made on the test datasets and compared to the labels using accuracy to assess correct classification, along with the classification report displaying precision, recall, and F1 score, and the confusion matrix illustrating true positives, true negatives, false positives, and false negatives.

5.7. Chapter Summary

This chapter discussed the project's design structure using the waterfall methodology, which demonstrated progressing through each stage to build the ransomware detection model. It provides in-depth details for each phase, such as the dataset collection, cleaning, augmentation, model training and implementation. The dataset was cleaned by removing irrelevant columns, encoding data for Machine Learning compatibility, and expanding it from 21,000 to 87,000 samples through data augmentation. The Random Forest Classifier was trained using different Python libraries, and split with an 80% training set and a 20% testing set.

The following chapter will discuss how the trained model will be implemented and used to simulate real-time detection with in-depth details.

SAMPLE THESIS

Chapter 6

Implementation

SAMPLE THESIS

6. Chapter Overview

The previous chapter discusses the design of the dataset, including dataset augmentation, the Random Forest algorithm, and the Machine Learning model that will be implemented for the detection of ransomware. It provides an in-depth design and the coding flow used to complete the system.

This chapter will cover the implementation phase of the ransomware detection system. To understand it, the technical aspects and coding involved will be explained. Then, it will describe how all components are integrated and how the system will be put into operation.

6.1. Real-Time Detection Simulation

As part of the systems implementation, a simulation was created to replicate how the trained ransomware detection system would operate in a real-time classification environment. The following code snippets will illustrate how the selected individual samples were used and tested within the model to assess its prediction of benign or malware.

```
sample_from_dataset = [17, 509, 1989, 2477, 5678]
sample_data = features_test.iloc[sample_from_dataset]
sample_labels = labels_test.iloc[sample_from_dataset]


sample_predictions = model.predict(sample_data)
```

Figure 6.1: Code snippet for model simulation of real-time detection.

```
for i, index in enumerate(sample_from_dataset):
    actual = "Benign" if sample_labels.iloc[i] == 0 else "Malware"
    predicted = "Benign" if sample_predictions[i] == 0 else "Malware"
    print(f"Sample {index} -> Actual: {actual} | Predicted: {predicted}")
```

Figure 6.2: Code snippet for model simulation of real-time detection.

Regarding Figure 6.1, the code involves manually selecting five different samples from the features_test set of the model. These samples are used because they are not part of the training set, making them unseen data for the model, which simulates the arrival

of new files on the system. When these samples are selected, they are then extracted from the labels and features using .iloc. Following this, after the selected samples have been extracted, the trained Random Forest will be used to predict these samples and classify them as benign or malware.

In Figure 6.2, the code shown is the loop used to display the results of the real-time detection simulation for each tested sample. Using enumerate(), the code will traverse each sample one at a time in the order they are set. For each sample, it will then be compared with the true class label from sample_labels and the predicted result from sample_prediction, and then be classified as either benign or malware based on the classification outcome. It will then print the message showing the classification results, thus simulating whether the model can be used for real-time detection.

This approach was used to test the model and see if it could successfully handle real-time inputs and accurately identify which samples were benign and malicious outside of the training set. By selecting these individual samples, the model would be required to make decisions on a file-by-file basis, providing a more detailed understanding of how the model operates and how it detects new threats to distinguish benign files. It will also simplify reporting, as the samples' index, true label, and predicted label will all be printed together, making it easier to understand and verify the model's success in classification. The implementation of real-time detection is vital for demonstrating the model's performance. While performance metrics such as accuracy, recall, and F1-score offer a statistical assessment, the implementation reveals the differences between evaluation and actual deployment. It demonstrates that the model is not merely a statistical tool but can also be applied effectively in a real-time environment where ransomware detection is essential.

GitHub link to designs - https://github.com/iaK-nosnhoj/MastersProject.git

6.2. Chapter Summary

This chapter has demonstrated and provided a detailed discussion of the implementation stage of the ransomware detection system, including a real-time detection simulation. The trained Random Forest model successfully classified unseen samples as benign or malware, showcasing its precision and effectiveness in real-time.

SAMPLE THESIS

Chapter 7

Testing

SAMPLE THESIS

7.1. Chapter Overview

The preceding chapter discusses the implementation of the ransomware detection system, including code snippets and concepts used to achieve the proposed solution.

The testing chapter employs the design phase of the dataset and the Random Forest algorithm configured for malware classification, presenting results that evaluate the success of detection rates.

Testing detection systems is essential to ensure they meet high standards and operate as intended. The accuracy and dependability of these systems can be assessed during various testing phases. Consequently, this chapter will examine different testing stages for different components of the detection model, along with the corresponding results, to help determine the success of the tests and if the model can be used as intended.

7.2. Indicators of Test Success

For the system to be a success, it must pass a specific criterion; otherwise, it will be considered a failure and must be re-evaluated.

7.2.1. Pass Criteria

- System must provide high accuracy over 99.00%
- High precision, recall, and F1 scores
- Well-balanced confusion matrix
- Correct classification predictions on real-time detection
- Correct running of code with no errors

7.2.2. Fail Criteria

- Code fails to run
- Accuracy lower than 99.00%
- Wrong classification predictions
- Low precision, recall, and F1 scores

7.3. Test Schedule

The testing of each component will be conducted on different dates and times, at various stages. The following schedule pertains to the entire detection system and its complete development.

SAMPLE THESIS

Table 7.1. Test schedule showing dates, times, and any comments noted

Date	Test	Comments
22/06/25	Original Dataset	Good overall accuracy scores
23/07/25	Augmented Dataset	Increased results from the last test
04/08/25	Real-Time Simulation	Great overall model and classifications

7.4. Original Dataset and Classification Testing

A series of tests was completed on the original dataset before augmentation for baseline performance of the Random Forest Classifier, as seen in Table 7.1 and Figure 7.1. The completed test was trained using 20 different random_state seeds in the train_test_split function and observed how the performance of each run varied with various seeds.

Each test involved an 80% training and 20% testing split, using the same trained models, and then recording all performance metrics, which included accuracy, true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN).

All 20 runs produced consistent and high-accuracy results, ranging from 99.17% to 99.63%, with only minimal variation. These results demonstrate that the model had strong baseline performance without dataset augmentation. The testing results also show that the true positive and true negative rates were high, providing confidence that the model can effectively differentiate and identify both benign and malware samples. However, during some runs, certain seeds exhibited slightly high false positive and false negative rates. This outcome justifies dataset augmentation, as it could improve the model's training and make the detection system more robust and accurate.

SAMPLE THESIS

Table 7.2: Accuracy results for the classification on the dataset before augmentation, including true positives, true negatives, false positives, and false negatives.

Test ID	Random State Seed	Accuracy	TP (True Positive)	TN (True Negative)	FP (False Positive)	FN (False Negative)
1	2	99.24%	2169	2146	30	6
2	4	99.47%	2172	2156	20	3
3	6	99.26%	2168	2151	25	7
4	8	99.35%	2168	2155	20	8
5	10	99.47%	2169	2159	16	7
6	15	99.40%	2168	2157	19	7
7	17	99.35%	2171	2152	23	5
8	19	99.49%	2175	2154	21	1
9	24	99.49%	2171	2158	18	4
10	29	99.31%	2169	2152	23	7
11	35	99.31%	2167	2154	21	9
12	37	99.40%	2173	2152	23	3
13	40	99.63%	2170	2165	11	5
14	42	99.17%	2169	2146	30	6
15	46	99.58%	2171	2162	13	5
16	50	99.58%	2171	2162	14	4
17	55	99.49%	2175	2154	21	2
18	63	99.42%	2173	2153	23	2
19	77	99.54%	2173	2158	17	3
20	89	99.49%	2173	2156	19	3

```
=== Evaluation ===
Accuracy: 0.9963226844403585

Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.99      1.00      2176
           1       0.99      1.00      1.00      2175

    accuracy                           1.00      4351
   macro avg       1.00      1.00      1.00      4351
weighted avg       1.00      1.00      1.00      4351


Confusion Matrix:
 [[2165   11]
 [   5 2170]]
```

Figure 7.1: Code output of results for Test ID 13 with a random state seed of 40, including accuracy, precision, recall, F1 score, and confusion matrix.

7.5. Augmented Dataset and Classification Testing

After the initial set of tests on the dataset, the data was augmented to artificially increase the number of samples and improve the overall accuracy of the model. The process involved duplicating the dataset three times and adding noise to each sample to generate new samples, resulting in a total dataset size of 87,008 samples. To access the new augmented dataset, the same testing process was completed to test the overall accuracy of the model. The augmented dataset was tested on an 80% training and 20% testing split, using the same trained models, and then recording all performance metrics, which included accuracy, true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN). The results showed consistent and higher accuracy than previous tests, ranging from 99.82% to 99.95%, an improvement from the peak of 99.63% in the results before augmentation. There were also high rates of true positives and true negatives across all 20 selected seeds, confirming the model's ability to distinguish between benign and malware samples, while false positives and false negatives decreased in most runs.

Table 7.3: Accuracy results for the classification on the dataset after augmentation, including true positives, true negatives, false positives, and false negatives.

Test ID	Random State Seed	Accuracy	TP (True Positive)	TN (True Negative)	FP (False Positive)	FN (False Negative)
1	2	99.82%	8695	8677	24	6
2	4	99.87%	8696	8685	16	5
3	6	99.86%	8690	8688	13	11
4	8	99.92%	8698	8691	10	3
5	10	99.93%	8698	8692	9	3
6	15	99.93%	8700	8691	10	1
7	17	99.89%	8698	8685	16	3
8	19	99.93%	8696	8695	6	5
9	24	99.89%	8693	8690	11	8
10	29	99.89%	8690	8694	7	11
11	35	99.95%	8698	8696	5	3
12	37	99.93%	8697	8693	8	4
13	40	99.89%	8695	8688	13	6
14	42	99.87%	8695	8686	15	6
15	46	99.92%	8699	8690	11	2
16	50	99.85%	8693	8684	17	8
17	55	99.92%	8698	8691	10	3
18	63	99.90%	8691	8694	7	10
19	77	99.89%	8695	8689	12	6
20	89	99.90%	8692	8693	8	9

```
=== Evaluation ===
Accuracy: 0.9995402827261234

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      8701
           1       1.00      1.00      1.00      8701

    accuracy                           1.00     17402
   macro avg       1.00      1.00      1.00     17402
weighted avg       1.00      1.00      1.00     17402


Confusion Matrix:
 [[8696    5]
 [   3 8698]]
```

Figure 7.2: Code output of results for Test ID 11 with a random state seed of 35, including accuracy, precision, recall, F1 score, and confusion matrix.

7.6. Real-Time Detection Simulation and Testing

Once the model had been fully trained and tested using the best seed, it was crucial to begin tests on real-time detection to validate the model's performance in real-world scenarios. To achieve this, a real-time simulation was conducted, which involved selecting five individual samples from the test set of the augmented dataset and then running them through the trained model one by one, reflecting how files would be scanned and classified in a live environment.

This test was conducted because, in real-life scenarios, ransomware needs to be detected as quickly as possible when files or applications are being scanned, or serious consequences can ensue. This testing aimed to address such behaviour and verify the prediction accuracy by testing on small sample sets while under real-time conditions. This test is important as it could highlight areas to where the model could be improved if any samples are misclassified.

Table 7.4: Prediction results for each sample run on test 1, including their classification predictions and actual classifications.

Test 1					
Sample	17	509	1989	2477	5678
Prediction	Benign	Malware	Malware	Malware	Malware
Actual	Benign	Malware	Malware	Malware	Malware

Table 7.5: Prediction results for each sample run on test 2, including their classification predictions and actual classifications.

Test 2					
Sample	72	857	2007	3475	7824
Prediction	Malware	Benign	Benign	Benign	Benign
Actual	Malware	Benign	Benign	Benign	Benign

Table 7.6: Prediction results for each sample run on test 3, including their classification predictions and actual classifications.

Test 3					
Sample	1458	6786	9001	5673	2222
Prediction	Malware	Benign	Benign	Malware	Malware
Actual	Malware	Benign	Benign	Malware	Malware

Table 7.7: Prediction results for each sample run on test 4, including their classification predictions and actual classifications.

Test 4					
Sample	9574	5823	7689	3687	6888
Prediction	Benign	Benign	Malware	Malware	Benign
Actual	Benign	Benign	Malware	Malware	Benign

Table 7.8: Prediction results for each sample run on test 5, including their classification predictions and actual classifications.

Test 5					
Sample	10679	14594	7375	3230	16589
Prediction	Malware	Benign	Benign	Benign	Benign
Actual	Malware	Benign	Benign	Benign	Benign

SAMPLE THESIS

7.7. Chapter Summary

This chapter evaluated the performance and accuracy of the Ransomware detection system through several completed tests using different seeds to measure the classification results. The model was first tested on the original dataset, achieving high accuracy rates between 99.17% and 99.63%, which established a strong baseline performance for the model. Following the completion of the dataset augmentation, it was subsequently tested again, which improved the accuracy to results between 99.82% and 99.95%, and also provided lower false positive and false negative results. The real-time detection simulation was also completed, replicating a live environment, and the results provided accurate predictions for all samples.

The following chapter will discuss the final results of the ransomware detection rates and explore their application in a real-world environment.

SAMPLE THESIS

Chapter 8

Results

SAMPLE THESIS

8.1. Chapter Overview

The previous chapter discussed the testing phase of the project, showing all stages of the model and different seeds used to help improve the overall performance of the model.

This chapter will present the overall results of the ransomware detection model. It will assess the success of all the tests conducted, offering insights and comparisons through graphs and charts to demonstrate the model's performance. It will examine each result in detail, discussing each component and explaining its function and significance.

8.2. Precision, Recall and F1 Score Results

In reference to Figures 8.1 and 8.2, both illustrate the results of the precision, recall, and F1-scores for both classifications using the best tested seeds before and after augmentation.

In each graph, the two classes have separate bars displaying the precision, recall, and F1-score results. The blue bar represents the precision, which indicates how accurately the model was able to identify ransomware and benign samples without errors, such as labelling regular files as malicious. Both Figures 8.1 and 8.2 display high-precision results, with values of 1.00 and 0.99, indicating the model produces very few false negatives and false positives. This builds confidence and assurance that the model can operate in a real-time environment with minimal misclassifications. It also demonstrates that after the dataset was augmented, the precision results improved, making it a better model with the newly created dataset.

Following this is the recall score, highlighted in orange. Recall, also known as the True Positive Rate, is a performance metric used to evaluate the effectiveness of the model and its ability to detect all actual ransomware samples in the dataset. High recall scores show that the model operates efficiently and accurately by successfully identifying all ransomware samples without issues. As seen in both Figures 8.1 and 8.2, all recall scores are 1.00 and 0.99, confirming that the model functions correctly and classifies most ransomware samples accurately. As indicated by the precision results in the chart, the scores increased after data augmentation; the same trend was observed with recall, which also improved once the newly created dataset was used on the model.

SAMPLE THESIS

The final result on the bar chart is the F1-score, which combines both the precision and recall scores into a single metric that provides a balanced and average measure of the model's overall performance.

Since both precision and recall results consistently showed high percentages across both classes for the best seeds before and after augmentation, the F1-scores also achieved high results for both classes. These outcomes demonstrate that the model can accurately classify ransomware while maintaining balance by minimising false positives and false negatives. The overall high F1-score reflects the model's performance, providing evidence that it could be utilised in real-time practical environments where accuracy and precision are crucial.

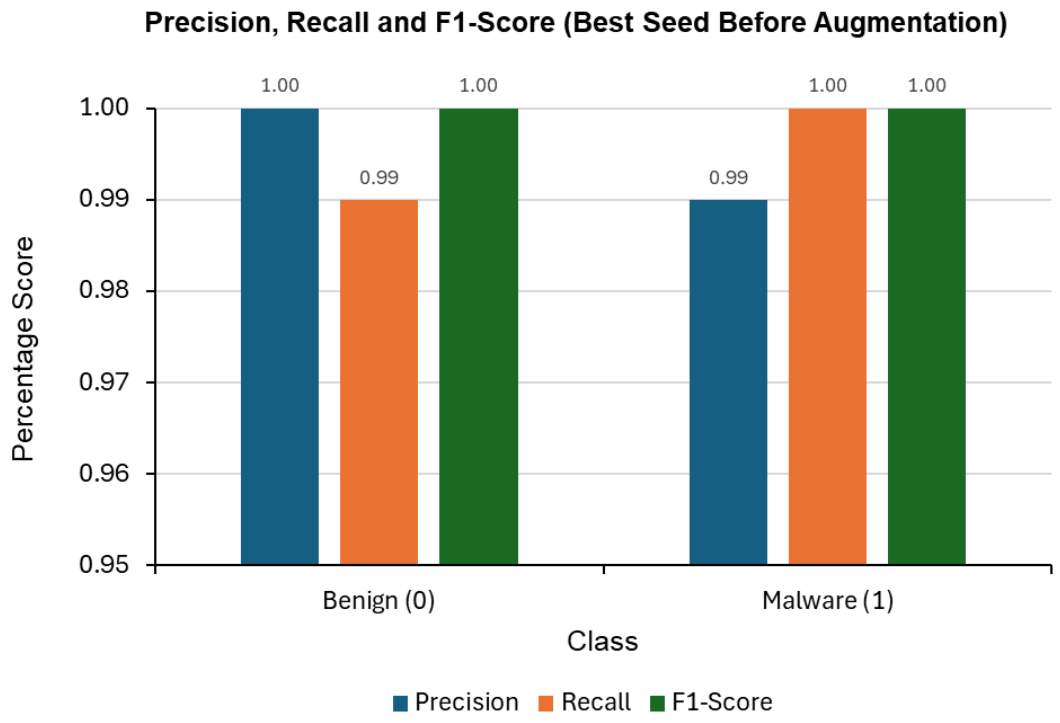

Figure 8.1: Graph illustrating the precision, recall, and F1 scores for both benign and malware classes, using the best tested seed before augmentation.

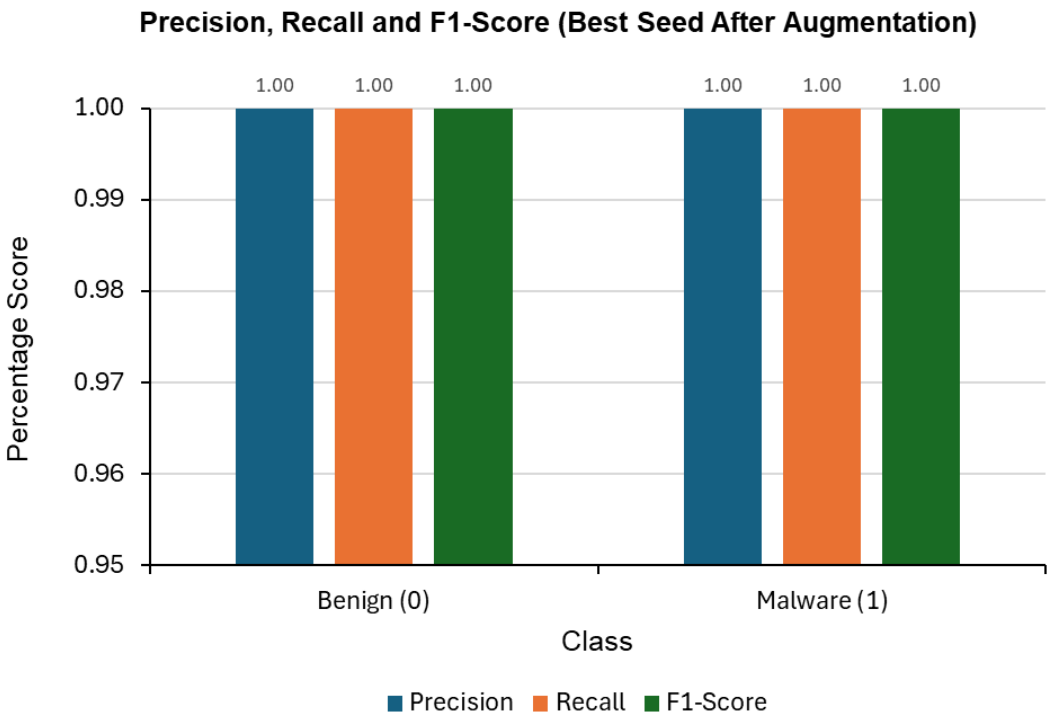

Figure 8.2: Graph illustrating the precision, recall, and F1 scores for both benign and malware classes, using the best tested seed after augmentation.

SAMPLE

8.3. Accuracy Trend Line Graph

The following line graph shows the accuracy trends of all 20 tested seeds before and after augmentation. The results reveal clear variation across the tested seeds, but they also show that the model trained with the augmented dataset consistently achieved higher accuracy, remaining above 99.8% in all tested runs.

In contrast to these results, the model before the dataset augmentation showed more variability in its results, with accuracy ranging from 99.20% to 99.60%, providing further evidence that the model performed better and more accurately with the augmented dataset. Overall, these results indicate that augmentation not only increased the accuracy of the models but also reduced the variance between tests, providing similar results for each seed. These improvements highlight the positive impact that dataset augmentation can have on the effectiveness of the Machine Learning model.

THESIS

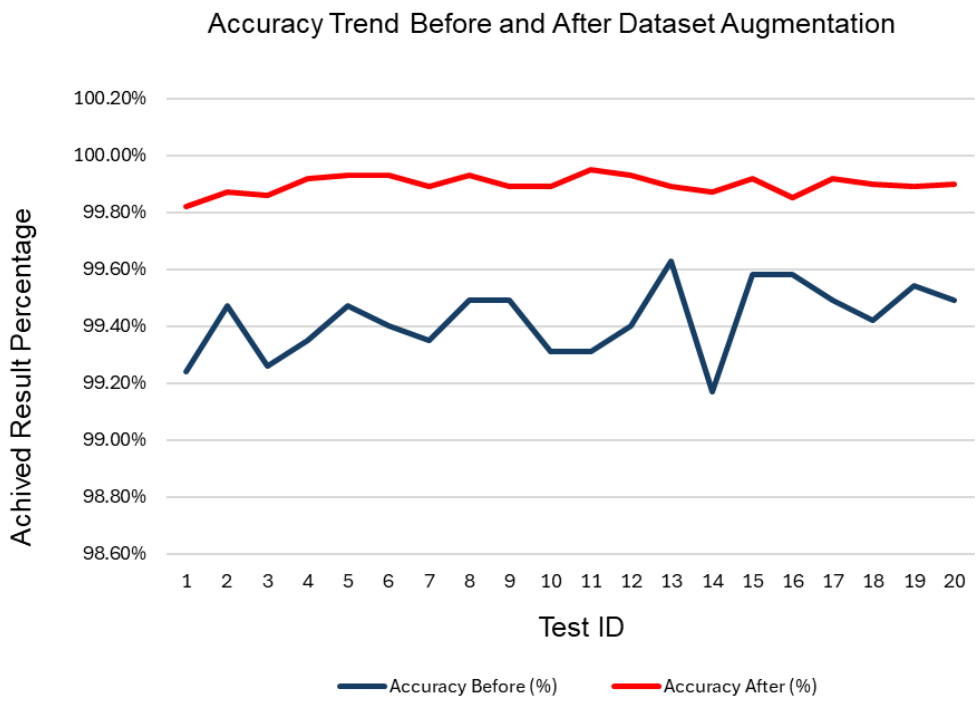

Figure 8.3: Accuracy trend before and after augmentation across all 20 tested seeds.

8.4. Accuracy and Seed Comparison

The bar chart in Figure 8.4 demonstrates the accuracy achieved across all 20 seeds, comparing the results of both datasets before and after augmentation. It is evident that the augmented dataset consistently outperformed the non-augmented dataset across all seeds, with accuracies of around 99.8% and 99.9%. In contrast, the standard dataset shows greater fluctuations and remains lower, around 99.2% and 99.6%. These comparisons further support the use of the augmented dataset, as it provides more stable results across all 20 seeds compared to the non-augmented dataset.

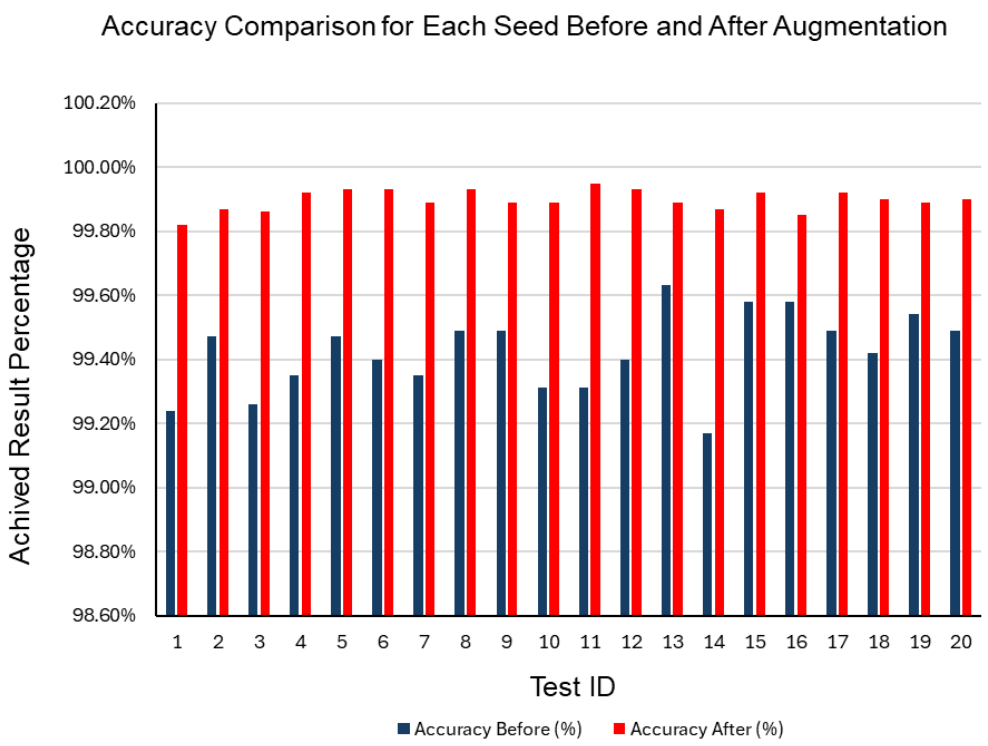

Figure 8.4: Accuracy comparison before and after augmentation across all 20 tested seeds.

8.5. Confusion Matrix Heatmaps

The next set of results presents the confusion matrix heatmaps, as shown in Figures 8.5 and 8.6. The confusion matrix heatmap visually represents the model's classification performance, displaying how many predictions fall into each category of the actual versus predicted labels. In the diagrams, the diagonal cells from top left to bottom right indicate correct classifications, while the off-diagonal cells from top right to bottom left highlight false positives and false negatives. The intensity and depth of the heatmap colours reflect the total number of samples, making it easier to recognise where the model performs well or where issues may arise.

Figure 8.5 shows the confusion matrix before dataset augmentation. It indicates that 2165 benign and 2170 samples were correctly classified, while 11 benign samples were misclassified as malware and 5 malware samples as benign. Although 4351 samples in total were correctly classified, the model only made a maximum of 16 errors, amounting to 0.4%, which provides a strong baseline for the model's classification ability. However, the small number of false positive classifications shows the model will occasionally misclassify benign samples, which could lead to unnecessary alerts for organisations in real-time environments.

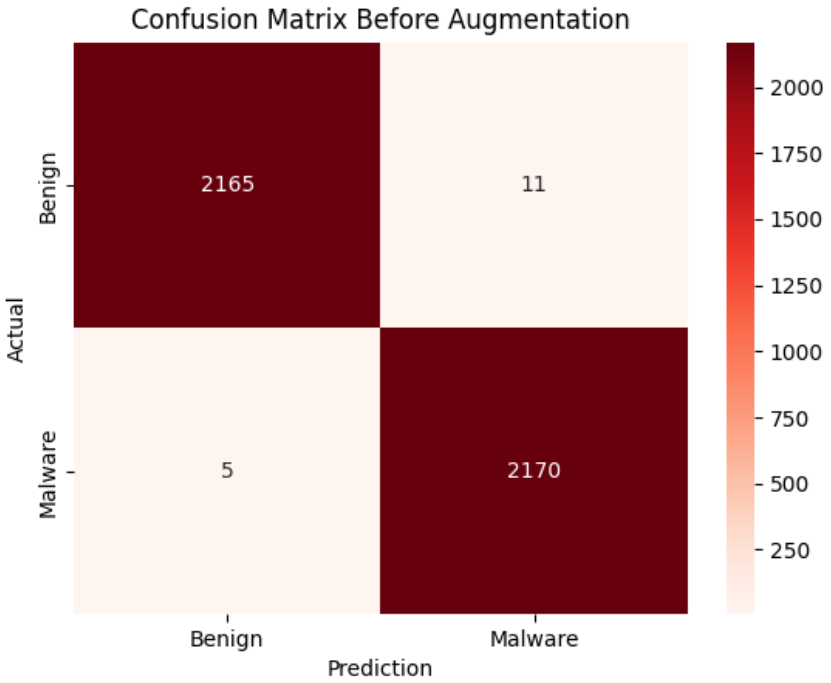

Figure 8.5: Confusion matrix heatmap before dataset augmentation.

The following confusion matrix heatmap, as shown in Figure 8.6, illustrates the classification results after dataset augmentation, clearly indicating a significant improvement from Figure 8.5. The model accurately classified 8696 benign samples and 8698 malware samples, with 5 false positives and only 3 false negatives out of a total of 17,402 samples. These results accumulate to an error rate of only 0.05%, showing the overall model's accuracy and precision while using the augmented dataset.

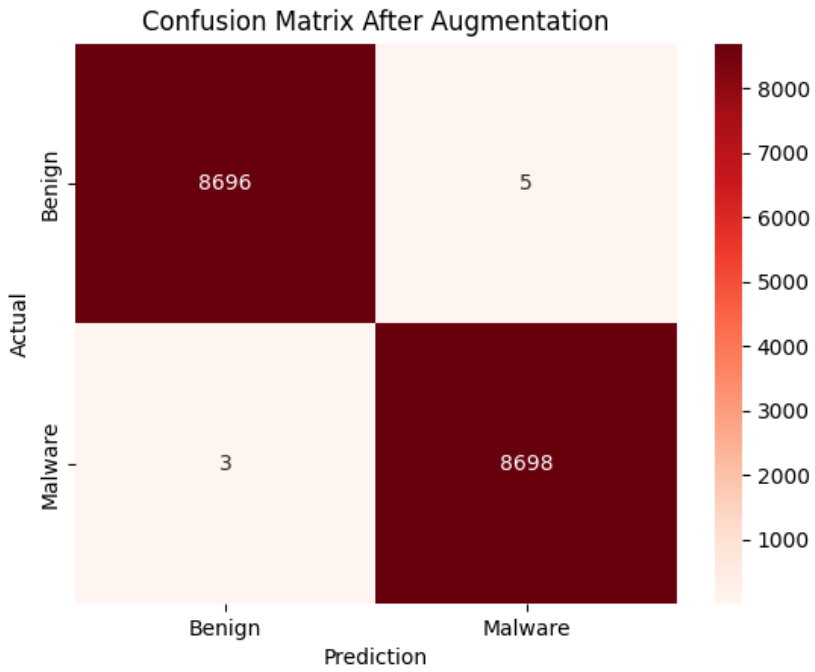

Figure 8.6: Accuracy comparison before and after augmentation across all 20 tested seeds.

8.6. Correctly Predicted Sample Charts

Lastly, the final results displayed in Figures 8.7 and 8.8 show pie charts highlighting the total number of correct predictions compared to incorrect predictions. Before dataset augmentation, out of 4,351 test samples, 4,335 were classified correctly, amounting to 99.6%, while 16 were misclassified, making up 0.4%. The pie chart is mostly dominated by the correct predictions segment, demonstrating how well the model performed before the use of the augmented dataset.

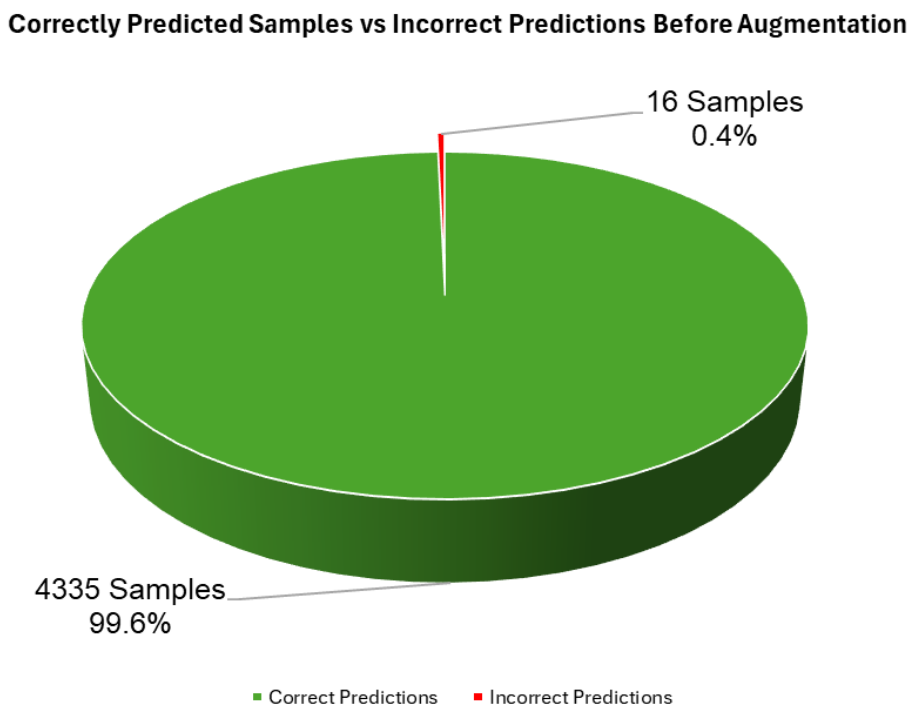

Figure 8.7: Pie chart showing correctly predicted samples versus incorrect samples before dataset augmentation.

When testing with the augmented dataset, a total of 17,402 samples were included. Of these, 17,394 were correctly classified by the model, resulting in 99.95%, with only 8 samples misclassified, accounting for 0.05%. The minimal error rates for misclassifications make it almost perfect in this case, with the pie chart showing a significantly larger green section compared to the other pie chart.

These comparisons between the two pie charts illustrate the effectiveness of dataset augmentation and how it has enhanced the overall model's ability to classify benign and malware samples. Although the model was already achieving high results before augmentation, the augmented dataset has improved accuracy and halved error rates. This demonstrates that the model has become more consistent, with few instances of misclassification.

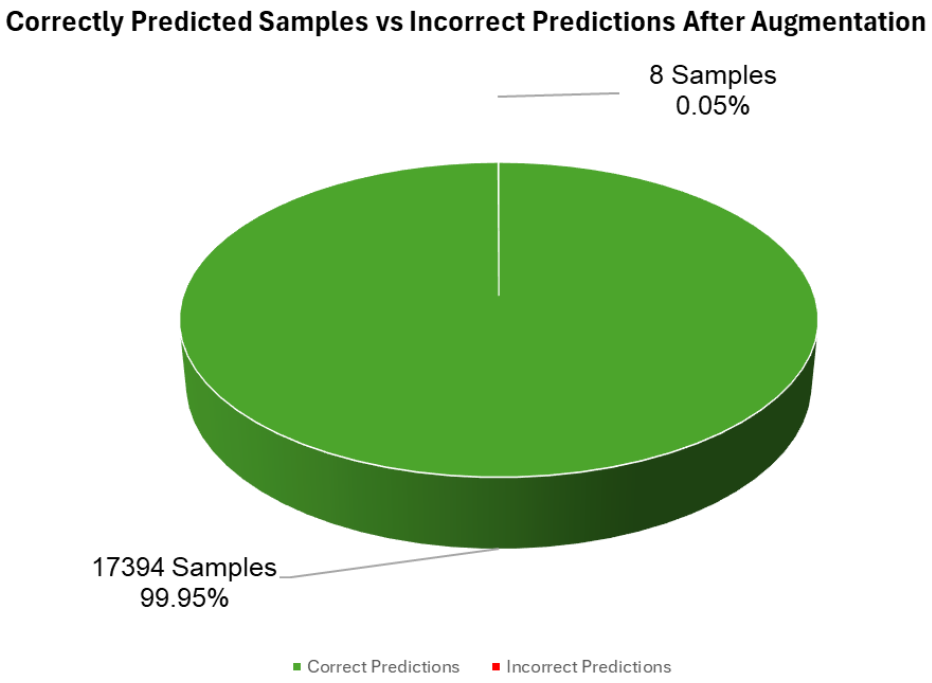

Figure 8.8: Pie chart showing correctly predicted samples versus incorrect samples after dataset augmentation.

8.7. Chapter Summary

This chapter has discussed the overall results of the ransomware detection model with the augmented dataset. It demonstrates strong performance across all evaluated metrics such as precision, recall, and F1-score. Both the accuracy trends and seed comparisons confirmed that using the augmented dataset increased all results above 99.8% and also reduced variation between tests. The confusion matrices and pie charts have also provided evidence of significantly low error rates in classification, from 0.4% before augmentation to 0.05% afterwards. Overall, using the augmented dataset has enhanced the model's accuracy, consistency, and reliability.

Chapter 9

Conclusion and Recommendations

SAMPLE THESIS

9.1. Chapter Overview

As the research project reaches its conclusion, this chapter will focus on the entire report and provide a summary. It will highlight the key achievements, the problems and issues that arose, the mitigation strategies used to overcome these challenges, and how the use of course materials contributed to the success of the project, followed by future recommendations that could be completed to further this project.

9.2. Key Achievements

The provided points emphasise the key achievements obtained during the progression and completion of the ransomware detection project.

- The overall project was a success, reaching the goals outlined at the start.
- Successfully developed my own augmented dataset, increasing it from 21,000 samples to 87,000 samples, which was essential for the functionality and high accuracy of the ransomware detection system.
- Successfully developed the Random Forest algorithm to process my dataset and accurately classify all samples tested.
- Achieved a high classification accuracy of 99.95%.
- Successfully overcame all challenges and issues that arose during the project, achieving complete success and completion of the model and project.

9.3. Limitations in the Current Model

The current model provides excellent classification results, correctly predicting and classifying all benign and malicious samples when tested. However, some limitations are evident in the model, and the author believes that extra criteria must be set to enhance the model for better development.

Currently, the model is limited to a simple real-time simulation for detecting ransomware to demonstrate its effectiveness and accurately classify samples during scanning. The author believes that more advanced real-time simulations can be developed to provide deeper insights into the model's performance and sample classification, thereby further enhancing the model.

SAMPLE THESIS

9.4. Challenges and Issues During Project

- The lack of knowledge in the programming field related to Machine Learning, Random Forest, and detection systems proved to be a significant challenge throughout the entire project. To overcome this challenge, several programming articles, journals, blogs, websites, and videos were studied to help learn the programming tools needed to complete this project. Doing so pushed the author to his limits and expanded his knowledge in the programming field, which helped with the project's progress and completion.
- During the project, the University of the West of Scotland locked all the authors' accounts and Banner IDs, thereby restricting access to all resources, contacts with a supervisor, OneDrive files, and emails. This continued for a total of 1.5 weeks, hindering progress on the project and ultimately halting it. This issue was solved by contacting student services a number of times, with no progress, then coming into the university and having an appointment with the IT services, where the problem was eventually solved. This challenge was mitigated by reviewing the model code to improve its performance and ensure smooth operation, while no progress could be made on the report.
- Creating a complete Machine Learning model with a new, recently developed dataset and a detailed report within a set timeframe proved to be a demanding and stressful task for the author, especially due to the time lost being locked out of university accounts. This challenge was overcome thanks to the guidance and invaluable advice provided by Dr. Althaff Irfan Cader Mohideen, who offered information and feedback to better structure the layout and contents of the final report, as well as valuable insights on developing the Machine Learning model.
- While completing the final report, the author relied on autosave when finishing parts of the report. An instance occurred when the autosave did not complete properly while exiting the document, ultimately causing the loss of part of the progress made when completing the report. This was mitigated by having multiple saved copies on the desktop, which helped preserve some of the work lost when closing the document.
- A significant challenge for the author during the project was their mental health and motivation. As the project faced several hurdles that needed to be overcome to complete it, along with personal issues that arose along the way, it had a profound

SAMPLE THESIS

impact. Support and encouragement from family and friends were essential in helping the author progress and complete the report and project.

9.5. Use of University and Course Materials

Although the author was not familiar with ransomware detection systems and Machine Learning, the knowledge provided by specific modules and individuals contributed to the project's success.

- The project was planned effectively by the author due to the knowledge gained through the Research Methods and Design module. Using various methods learned from the module, such as the Gantt Chart and project management techniques, to facilitate the flow of the project, as well as resources like the sample specification form to guide the project and brainstorming ideas for the project title.
- The valuable information in the Master's Project module assisted the author with the final report. The project handbook offered detailed guidance on what is required in the report, what should be included, and the layout in which it should be submitted.
- The practical knowledge gained from the programming module helped with the code development stages of the project and assisted in the software selection used for the project.
- The knowledge gained from the Threat Intelligence module enhanced the understanding of ransomware, including its problems, background, and the need for detection systems to prevent further attacks.
- Knowledge and insight gained from Dr. Althaff Irfan Cader Mohideen contributed to the project's progress and guided the next steps I could take to enhance the model further.

9.6. Future Recommendations

- Expanding the dataset with more real-world ransomware would help the model recognise newer, unseen, and zero-day variants better and enhance its overall generalisation.
- Develop a more sophisticated real-time detection approach, as the current method relies on simulating through individual samples. Future suggestions

SAMPLE THESIS

include integrating live file system tools to classify files as they appear. An application could also be created to enable users to scan files immediately as they emerge on the system.

- The model could be tested in cloud-based environments further to enhance the model in different areas of cybersecurity.

9.8. Chapter Summary

This chapter reflects on the successful completion of the ransomware detection project, highlighting several key achievements, such as creating a new augmented dataset, achieving 99.95% classification accuracy, and developing a robust Random Forest model.

During the completion of the project, several challenges arose, such as limited programming knowledge, account access problems, mental health issues, and time constraints, all of which were overcome through strong support from friends and family and ongoing self-learning. Also, several university modules and course resources played a crucial role in completing the project, including the structure and content required.

SAMPLE THESIS

Chapter 10

Self-Evaluation

SAMPLE THESIS

10.1. Chapter Overview

As the final part of the project report, the author must complete a self-evaluation to reflect on their learning, performance, and growth throughout the project. The chapter will discuss all the technical skills the author has developed, as well as the project management and time management involved, the communication and feedback received, personal developments and areas for improvement for the future.

10.2. Technical Skill Development

One of the main areas of growth for me during this project was improving my technical skills, especially in Machine Learning, Python programming, and data preprocessing. Before starting this project, my understanding of machine learning and its applications was limited. Although I had some experience in programming and ransomware through university modules, I had never developed or implemented a complete machine learning model.

Throughout this project, I studied Python programming, developing a strong knowledge of several key data science libraries, including pandas, NumPy, and scikit-learn, which were essential for completing my project. These libraries were used in various components of my project, from data cleaning to model training. Learning these libraries was necessary as I would not have been able to prepare the dataset and use it on the Random Forest algorithm without a complete dataset. Following this, I also learnt about the Random Forest Classifier, which I successfully integrated into my project. This also helped me gain experience with model training and evaluation metrics. I also explored dataset augmentation to expand on the dataset used in my model, which provided me with further knowledge in programming. Lastly, I created a simple real-time detection simulation to classify individual samples from my dataset, demonstrating that the Machine Learning model was functioning. This helped me understand how the models can operate in real-time.

10.3. Project Management and Time Management

For all projects, effective project and time management is essential to ensure successful completion. Without this, all plans and projects could fail due to a single problem arising. For my project, I followed the waterfall methodology due to its step-by-step process, which aligned with my plan as I moved through each stage, such as

SAMPLE THESIS

planning, data collection, data augmentation, model training, simulation, testing, and evaluation.

To assist with this process, I used a pre-planned Gantt chart to guide my schedule, allowing me to set clear goals and deadlines to track progress. However, as with most projects, unexpected problems did occur, such as being locked out of university accounts for over a week, illness and personal issues. Although time management was challenging at certain points in the project, breaking it down into sections and phases greatly helped me stay focused. Thanks to this project, I have improved my project and time management skills and can apply them to future projects I might be involved in.

10.5. Communications and Feedback

The communication and feedback with Dr. Althaff Irfan Cader Mohideen were another essential part of my project and played a significant role in shaping its direction and quality, including the final report. Regular meetings via Teams with my supervisor helped me implement important changes to my design, enhancing the structure and clarity of my final report.

I learned to accept feedback when given and to implement it constructively and proactively. I also learned to ask as many questions as possible, to cover all bases, and not to stay quiet. This was important for me, as I don't usually ask questions and seek help or feedback.

The experience of receiving regular feedback and attending meetings has enhanced my overall ability to communicate more effectively with individuals and to ask for help when needed. Skills that will be essential in a professional and working environment.

10.6. Personal Development

Overall, the project has significantly enhanced my personal growth, particularly in managing long-term projects, working independently, and maintaining self-discipline to complete all the phases of my project. It also helped me to try and stay motivated to complete tasks after problems such as account lockouts, data loss, and setbacks from personal issues. In addition to this, it has helped me grow and improve my report writing skills, strengthen my time management and organisational skills from juggling my part-time job and completing the report.

SAMPLE THESIS

However, the support from my supervisor, family, and friends was crucial in helping me stay sane and motivated to complete this project, as well as boosting my confidence in working independently and strengthening my patience to overcome obstacles.

10.7. Areas for Improvement

Although the project was a success, there were definitely areas for improvement that could benefit future efforts. One key area I can improve on is time management or focus, as I occasionally found myself distracted by playing the PlayStation 5, which often impacted my productivity during essential stages of the project. Also, being distracted by my phone and watching reels on Instagram would severely divert me from my project. In future projects that I may participate in, I will aim to set better boundaries and create a more focused environment to prevent me from being distracted by my PS5 and phone, which will help me maximise my working efficiency.

10.8. Chapter Summary

This chapter has highlighted the key areas of my personal and technical growth that have been achieved throughout this project. It demonstrates I have developed stronger skills in Python programming, machine learning, and data preprocessing, while also improving my time management and communication skills.

All challenges such as my limited knowledge, access issues, and distractions which tested my focus were ultimately overcome through my motivation, support, and self-learning. This project has ultimately pushed me to my limits and will forever be remembered for helping me grow as an individual and professional.

SAMPLE THESIS

Chapter 11

References

SAMPLE THESIS

1. Adi Bleih (2025). *Ransomware Annual Report 2024*. [online] Cyberint. Available at: https://cyberint.com/blog/research/ransomware-annual-report-2024/. [Accessed 12 Jun. 2025].
2. Adnan Kutay Yüksel (2025). *Malware Detection Using Machine Learning Methods on the APIMDS Dataset-1: Preparation of the Dataset*. [online] Medium. Available at: https://medium.com/ai-genai-llm/malware-detection-using-machine-learning-methods-on-the-apimds-dataset-1-preparation-of-the-f683adcd8441 [Accessed 25 Jun. 2025].
3. Albishry, N., AlGhamdi, R., Almalawi, A., Khan, A.I., Kshirsagar, P.R. and BaruDebtera, U. (2022). An Attribute Extraction for Automated Malware Attack Classification and Detection Using Soft Computing Techniques. *Computational Intelligence and Neuroscience*, 2022, p.e5061059. doi:https://doi.org/10.1155/2022/5061059.
4. Alraizza, A. and Algarni, A. (2023). Ransomware Detection Using Machine Learning: A Survey. *Big Data and Cognitive Computing*, [online] 7(3), p.143. doi:https://doi.org/10.3390/bdcc7030143.
5. Association for Project Management (2023). *What Is Project Management?* [online] Apm.org.uk. Available at: https://www.apm.org.uk/resources/what-is-project-management/.
6. Baker, K. (2025). *5 Most Common Types of Ransomware | CrowdStrike*. [online] Crowdstrike.com. Available at: https://www.crowdstrike.com/en-gb/cybersecurity-101/ransomware/types-of-ransomware/ [Accessed 28 Jul. 2025].
7. Beaman, C., Barkworth, A., Akande, T.D., Hakak, S. and Khan, M.K. (2021). Ransomware: Recent advances, analysis, challenges and future research directions. *Computers & Security*, 111(1). doi:https://doi.org/10.1016/j.cose.2021.102490.
8. Belcic, I. and Stryker, C. (2021). *Supervised Learning*. [online] Ibm.com. Available at: https://www.ibm.com/think/topics/supervised-learning. [Accessed 20 Jun. 2025].
9. Bhushan, B. (2024). *Ransomware Families and Attacks: Comprehensive Guide*. [online] Stellar Data Recovery Inc. Available at: https://www.stellarinfo.com/article/ransomware-families-part1.php [Accessed 28 Jul. 2025].

SAMPLE THESIS

10. Chainalysis (2024). *Ransomware hit $1 billion in 2023*. [online] Chainalysis. Available at: https://www.chainalysis.com/blog/ransomware-2024/.
11. Dixit, A. and Singh, S. (2023). Malware Detection Using Random Forest. doi:https://doi.org/10.1109/icccnt56998.2023.10306628.
12. GeeksforGeeks (2018). *ML | Overview of Data Cleaning*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/data-analysis/data-cleansing-introduction/.
13. GOV (2025). *The experiences and impacts of ransomware attacks on individuals and organisations*. [online] GOV.UK. Available at: https://www.gov.uk/government/publications/the-experiences-and-impact-of-ransomware-attacks-on-victims/the-experiences-and-impacts-of-ransomware-attacks-on-individuals-and-organisations. [Accessed 14 Jun. 2025].
14. IBM (2021). *Unsupervised Learning*. [online] Ibm.com. Available at: https://www.ibm.com/think/topics/unsupervised-learning. [Accessed 20 Jun. 2025]. [Accessed 20 Jun. 2025].
15. IBM (2023). *Semi-Supervised Learning*. [online] Ibm.com. Available at: https://www.ibm.com/think/topics/semi-supervised-learning. [Accessed 20 Jun. 2025].
16. JPMorgan (2023). *The Potential Impacts of Ransomware*. [online] www.jpmorgan.com. Available at: https://www.jpmorgan.com/technology/news/the-potential-impacts-of-ransomware.
17. Khammas, B.M. (2020). Ransomware detection using random forest technique. *ICT Express*, 6(4). doi:https://doi.org/10.1016/j.icte.2020.11.001.
18. Masum, M., Hossain Faruk, M.J., Shahriar, H., Qian, K., Lo, D. and Adnan, M.I. (2022). *Ransomware Classification and Detection With Machine Learning Algorithms*. [online] IEEE Xplore. doi:https://doi.org/10.1109/CCWC54503.2022.9720869.
19. Murphy, D. (2023). *Common Ransomware Encryption Techniques*. [online] Lepide Blog: A Guide to IT Security, Compliance and IT Operations. Available at: https://www.lepide.com/blog/common-ransomware-encryption-techniques/.
20. O'Kane, P., Sezer, S. and Carlin, D. (2018). Evolution of ransomware. *IET Networks*, [online] 7(5), pp.321–327. doi:https://doi.org/10.1049/iet-net.2017.0207.

SAMPLE THESIS

21. Razaulla, S., Fachkha, C., Markarian, C., Gawanmeh, A., Mansoor, W., Fung, B.C.M. and Assi, C. (2023). The Age of Ransomware: A Survey on the Evolution, Taxonomy, and Research Directions. *IEEE Access*, [online] 11, pp.40698–40723. doi:https://doi.org/10.1109/ACCESS.2023.3268535.
22. Reddy, S., K. Sai Kumar, Siva, E. and Pothuraju RajaRajeswari (2023). Enhancing CyberSecurity Through Random Forests: A Comprehensive Analysis of Malware Detection and Intrusion Detection. doi:https://doi.org/10.1109/icuis60567.2023.00085.
23. Rele, M., Samuel, J., Patil, D. and Krishnan, U. (2025). Exploring Ransomware Detection Based on Artificial Intelligence and Machine Learning. *Procedia Computer Science*, [online] 252, pp.548–556. doi:https://doi.org/10.1016/j.procs.2025.01.014.
24. Reshmi (2021). Information security breaches due to ransomware attacks - a systematic literature review. *International journal of information management data insights*, [online] 1(2). doi:https://doi.org/10.1016/j.jjimei.2021.100013.
25. Saad, S., Briguglio, W. and Elmiligi, H. (2019). The Curious Case of Machine Learning In Malware Detection. doi:https://doi.org/10.48550/arxiv.1905.07573.
26. Sarker, I.H. (2021). Machine Learning: Algorithms, Real-World Applications and Research Directions. *SN Computer Science*, [online] 2(3), pp.1–21. Available at: https://link.springer.com/article/10.1007/s42979-021-00592-x.
27. SentinelOne (2025). *7 Types of Ransomware Attacks in 2025*. [online] SentinelOne. Available at: https://www.sentinelone.com/cybersecurity-101/cybersecurity/types-of-ransomware/#7-types-of-ransomware-attacks [Accessed 28 Jul. 2025].
28. Team EMB (2024). *Why Waterfall Methodology Remains Relevant in Modern Projects*. [online] EMB Blogs. Available at: https://blog.emb.global/waterfall-methodology-remains-relevant-in-modern-projects/.
29. UMATechnology (2024). *7 Common Endpoint Security Challenges and How to Overcome Them - UMA Technology*. [online] UMA Technology. Available at: https://umatechnology.org/7-common-endpoint-security-challenges-and-how-to-overcome-them/ [Accessed 23 Apr. 2025].
30. Yan, P. and Khoei, T.T. (2025). Securing the Internet of Things: A Comprehensive Review of Ransomware Attacks, Detection, Countermeasures,

SAMPLE THESIS

and Future Prospects. *Franklin Open*, [online] 11, p.100256. doi:https://doi.org/10.1016/j.fraope.2025.100256.

31. Yeap (2024). Council Post: Why Antivirus Signatures And Next-Gen Malware Detection Aren't Enough. *Forbes*. [online] 12 Aug. Available at: https://www.forbes.com/councils/forbestechcouncil/2021/09/03/why-antivirus-signatures-and-next-gen-malware-detection-arent-enough/.

SAMPLE THESIS

Chapter 12

Bibliography

SAMPLE THESIS

1. Adnan Kutay Yüksel (2025a). *Malware Detection Using Machine Learning Methods on the APIMDS Dataset-1: Preparation of the Dataset*. [online] Medium. Available at: https://medium.com/ai-genai-llm/malware-detection-using-machine-learning-methods-on-the-apimds-dataset-1-preparation-of-the-f683adcd8441.
2. Adnan Kutay Yüksel (2025b). *Malware Detection Using Machine Learning Methods on the APIMDS Dataset-6: Random Forest Algorithm*. [online] Medium. Available at: https://medium.com/ai-genai-llm/malware-detection-using-machine-learning-methods-on-the-apimds-dataset-6-random-forest-algorithm-5848399ce12c.
3. Awan, A.A. (2022). *A Complete Guide to Data Augmentation*. [online] DataCamp. Available at: https://www.datacamp.com/tutorial/complete-guide-data-augmentation.
4. GeeksforGeeks (2017). *ML | Data Preprocessing in Python*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/machine-learning/data-preprocessing-machine-learning-python/.
5. GeeksforGeeks (2019a). *One Hot Encoding in Machine Learning*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/machine-learning/ml-one-hot-encoding/.
6. GeeksforGeeks (2019b). *Python program to convert hex string to decimal*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/python/python-program-to-convert-hex-string-to-integer/ [Accessed 31 Jul. 2025].
7. GeeksforGeeks (2024). *Random Forest Algorithm in Machine Learning*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/machine-learning/random-forest-algorithm-in-machine-learning/.
8. Michał Marcińczuk (2024). *Heatmap for Confusion Matrix in Python | Towards Data Science*. [online] Towards Data Science. Available at: https://towardsdatascience.com/heatmap-for-confusion-matrix-in-python-20a9fc689665/.
9. SciKit Learn (n.d.). *sklearn.ensemble.RandomForestClassifier*. [online] scikit-learn. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html#.

SAMPLE THESIS

10. Shafi, A. (2023). *Sklearn Random Forest Classifiers in Python Tutorial*. [online] www.datacamp.com. Available at: https://www.datacamp.com/tutorial/random-forests-classifier-python.
11. Shittu Olumide (2025). *Data Cleaning Essentials with Pandas*. [online] Statology. Available at: https://www.statology.org/data-cleaning-essentials-with-pandas/ [Accessed 31 Jul. 2025].
12. Somanna, M.S. (2023). *Guide to Adding Noise to your Data using Python and Numpy*. [online] Medium. Available at: https://medium.com/@ms_somanna/guide-to-adding-noise-to-your-data-using-python-and-numpy-c8be815df524.
13. T, D. (2019). *Confusion Matrix Visualization*. [online] Medium. Available at: https://medium.com/@dtuk81/confusion-matrix-visualization-fc31e3f30fea.
14 w3schools (n.d.). *Pandas - Cleaning Data*. [online] www.w3schools.com. Available at: https://www.w3schools.com/python/pandas/pandas_cleaning_wrong_data.asp .

SAMPLE THESIS

Chapter 13

Appendix

SAMPLE THESIS

13.1. Project Specification Form

University of the West of Scotland

School of Computing, Engineering and Physical Sciences

MEng/MSc Cyber Security Project Specification

Student name: Kai Johnson

Banner ID: B00374253/B00382170

Email: B00374253@studentmail.uws.ac.uk

Project being undertaken on a part-time or full-time basis: Full-time

MEng/MSc Programme/stream:

MEng/MSc Programme Leader:

Project Title:

Enhancing the Efficacy of Ransomware Detection Systems Using the Random Forest Algorithm and Implementing Real-Time Endpoint Protection.

Research Question to be answered:

Can ransomware detection systems using the Random Forest algorithm have increased real-time protection security?

Outline (overview) and overall aim of project:

This research aims to design, develop, and evaluate a ransomware detection system utilising the Random Forest algorithm, aiming to ascertain its viability for real-time endpoint protection to enhance security. Furthermore, it seeks to assess the efficacy of the Machine Learning model in distinguishing between malicious and benign samples, as well as its potential integration into an operational security environment. The objective of this project is to provide and contribute to research that aids in the improvement of intelligent threat detection systems by delivering a functional

SAMPLE THESIS

detection system characterised by high detection accuracy and real-time responsiveness.

Objectives (list of tasks to be undertaken to achieve the overall aim of the project and to answer the research question posed):

Literature Review: Conduct a thorough critical analysis of the existing ransomware detection systems that employ Machine Learning or artificial intelligence methodologies, the Random Forest algorithm, and real-time endpoint protection techniques.

Dataset Collection and Preprocessing: Distinguish and gather relevant dataset for the Machine Learning model, such as infected ransomware files and benign samples. Prepare data for the Machine Learning model by cleaning, labelling and preprocessing it.

Promising dataset for project: Ransomware Dataset 2024 – Up to date and includes 21,752 files (10,876 malicious and 10,876 benign) and 26 distinct malware families, with a strong focus on ransomware.

Model Design and Training: Develop a Machine Learning model utilising PyCharm and implement the Random Forest algorithm through Python-based tools, including scikit-learn. The Machine Learning model will undergo training with the pre-processed dataset and will be enhanced through hyperparameter tuning.

Model evaluation: Evaluate the model's performance using several metrics such as accuracy, precision, ROC-AUC and F1 scores.

Simulation of real-time detection: Explore how the Machine Learning model can be used in a virtual real-time environment using open-source tools.

Documentation and analysis: Throughout the project, record all notes, progress, findings, problems that arise, and collected data. Also, suggest areas for future work, such as system integration and scalability.

Final report: Prepare the final report, including all the recorded findings, results, and conclusions of the completed work.

CyBOK Knowledge Areas (Please check at least two relevant areas your project going to be covered):

SAMPLE THESIS

Human, Organisational & Regulatory Aspects	☐	Risk Management & Governance
	☐	Law & Regulation
	☐	Human Factors
	☐	Privacy & Online Rights
Attacks & Defences	☒	Malware & Attack Technologies
	☐	Adversarial Behaviours
	☐	Security Operations & Incident Management
	☐	Forensics
Systems Security	☐	Cryptography
	☐	Operating Systems & Virtualisation Security
	☐	Distributed Systems Security
	☐	Formal Methods for Security
	☐	Authentication, Authorisation & Accountability
Software and Platform Security	☒	Software Security
	☐	Web & Mobile Security
	☐	Secure Software Lifecycle
Infrastructure Security	☐	Applied Cryptography
	☐	Network Security
	☒	Hardware Security
	☐	Cyber Physical Systems
	☐	Physical Layer and Telecommunications Security

SAMPLE THESIS

Indicative reading list (references to be correctly presented) and resources:

Dataset: Ransomware Dataset 2024
Programming Software: PyCharm

Adi Bleih (2025). *Ransomware Annual Report 2024*. [online] Cyberint. Available at: https://cyberint.com/blog/research/ransomware-annual-report-2024/.

Adnan Kutay Yüksel (2025). *Malware Detection Using Machine Learning Methods on the APIMDS Dataset-6: Random Forest Algorithm*. [online] Medium. Available at:

https://medium.com/ai-genai-llm/malware-detection-using-machine-learning-methods-on-the-apimds-dataset-6-random-forest-algorithm-5848399ce12c.

Alraizza, A. and Algarni, A. (2023). Ransomware Detection Using Machine Learning: A Survey. *Big Data and Cognitive Computing*, [online] 7(3), p.143. doi:https://doi.org/10.3390/bdcc7030143.

Ayeni, O.A. (2022). A Supervised Machine Learning Algorithm for Detecting Malware. *Journal of Internet Technology and Secured Transactions*, 10(1), pp.764–769. doi:https://doi.org/10.20533/jitst.2046.3723.2022.0094.

Azeem, M., Khan, D., Iftikhar, S., Shaikhan Bawazeer and Alzahrani, M. (2024). Analyzing and comparing the effectiveness of malware detection: A study of Machine Learning approaches. *Heliyon*, 10(1), pp.e23574–e23574. doi:https://doi.org/10.1016/j.heliyon.2023.e23574.

Dixit, A. and Singh, S. (2023). Malware Detection Using Random Forest. doi:https://doi.org/10.1109/icccnt56998.2023.10306628.

Kumar P, N.S., Singiri, V.P., Perapogu, S., Kunam, Y. and Mallela, V.K. (2024). Enhanced Malware Detection Using Machine Learning Algorithms. *IJARCCE*, [online] 13(4). doi:https://doi.org/10.17148/ijarcce.2024.13422.

Marking scheme:

Introduction	**5%**
Literature review	**15%**
Research Design/Methodology	**10%**
Design and Implementation	**20%**
Testing	**10%**
Results and Analysis	**10%**
Conclusions and recommendations	**10%**
Critical self-evaluation	**10%**
Structure, Style, and Referencing	**10%**

Supervisor:

Althaff Irfan Cader Mohideen

Moderator:

Raman Singh

Programme Leader:

Daune West

Date specification submitted:

22/05/25

Please complete the 'ethics' form below for all projects.

School of Computing, Engineering and Physical Sciences

MSc PROJECT – REQUIREMENT FOR ETHICAL APPROVAL

SECTION 1: TO BE COMPLETED BY THE STUDENT

Please answer the following questions:	Select Yes or No
Does your project involve: human subjects (e.g. as clients, providers of requirements, 'testers'/users), access to unpublished company documents/records, private personal data, questionnaires, surveys, focus groups and/or other interview techniques?	No
If your answer above is **Yes**, have you discussed with your supervisor and applied for ethical approval using the university's Ethical Review Manager online system?	Choose an item.
If you haven't applied for the ethical approval, have you started preparing necessary documents required for the ethical approval, e.g. information sheet, consent form, questionnaires, etc?	Choose an item.
I confirm that I will not start the above mentioned project work before obtaining formal ethical approval.	Yes

SAMPLE THESIS

If answering yes to the first question above, and if you have not done so already, please prepare and submit an application for ethical approval through the university's "Ethical Review Manager" (ERM) system, which is available at https://uws.forms.ethicalreviewmanager.com.

Before doing so, please consult the ERM applicant user guide, which can be downloaded from: https://www.uws.ac.uk/media/5364/erm-applicant-user-guide.pdf

The process for making applications and the principles applied in the School of CEPS are contained within the Ethical Approval Process. Please read this document before making an application for ethical approval.

Please provide any additional information, comments and suggestions:

All work will be completed under secure environments and all data collected will be publicly available.

Name of Student (Print name): Kai Johnson

Signature: K Johnson

Date: 22/05/25

SECTION 2: TO BE COMPLETED BY THE PROJECT SUPERVISOR

I understand that the above project requires/does not require* ethical approval (*please delete as appropriate).

Supervisor (print name):

Signature:

Date:

IMPORTANT: please note that by signing this form all signatories are confirming that any potential ethical issues have been considered and, where necessary, an application for ethical approval has been/will be made via the Ethical Review Manager software.

Any project requiring ethical approval, but which has not been given approval will not be accepted for marking.

Ethical approval cannot be sought in retrospect.

SAMPLE THESIS

13.2. Process Documentation

Kai Johnson (B00374253/B00382170) – Logbook, diary, and recorded documents/details.

To-do list – Project specification		
Date	**Details**	**Outcome**
22/04/25	Email Mohideen with my project idea and ask if he would be willing to be the project supervisor.	Thought it was a good idea and accepted to become my supervisor. He also recommended Raman as moderator.
22/04/25	Email Raman and ask if he would be willing to be the project moderator.	Accepted to become the project moderator.
21/05/25	Complete the project specification and email it to Mohideen and Daune.	Complete.
21/05/25	Discuss the marking scheme with Mohideen and ask about the details of the ethical approval.	Agreed on the marking scheme below: **Introduction** 5% **Literature review** 15% **Research Design/Methodology** 10% **Design and Implementation** 20% **Testing** 10% **Results and Analysis** 10% **Conclusions and recommendations** 10% **Critical self-evaluation** 10% **Structure, Style, and Referencing** 10%
22/05/25	Look for project resources and references.	Found resources needed and references.
22/05/25	Email specification to Mohideen and Daune.	Complete

To-do list – Interim Report		
Date	**Details**	**Outcome**
3/05/25	Complete Intro for report	Complete
12/06/25	Complete literature review for the report	Complete
04/07/25	Start the research design and methodology section for the report	Complete
11/07/25	Submission due	Complete – submitted 11th July

SAMPLE THESIS

MSc PROJECT (COMP11024)

PROJECT PROCESS DOCUMENTATION NUMBER 1

Student: **Kai Johnson** **Supervisor:** **Dr. Althaff Irfan Cader Mohideen**

Meeting Number: **1** **Date/Time: 11/07/2025**

Agenda for meeting:

Discussing the interim report and feedback

Discussion of agenda items:

- Discussed the interim report and provided feedback via Microsoft Teams.
- Lots of feedback on the general report, mentioned that the abstract and acknowledgements did not need chapter numbers.
- Enhance the header and footer by including the course name and attractive numbering.
- Each chapter has its own page with a larger font and has been made more presentable.
- Mentioned the current dataset being used was too small, and I should augment it to make the model better.
- Overall good start to the report

Summary of agreed action plan:

Made sure to augment the dataset to at least 80,000 samples, and continue working on the report as planned.

Notes: Meeting time 33 minutes, next meeting on the 25th July.

SAMPLE THESIS

To-do list – Dataset & Model Development		
Date	Details	Outcome
22/05/25	Acquired and downloaded the dataset	Downloaded the dataset and checked to make sure everything was in order and could be used for the model – All good
15/06/25	Start dataset cleaning on PyCharm, multiple cleaning processes need to be done to make it a viable dataset for the model to use	Researched ways to clean the dataset
15/06/25	Need to remove all irrelevant data.	Multiple sources found with correct use of code. Applied it and successfully removed irrelevant data.
16/06/25	Change class labels to numeric so the model can correctly classify the samples.	Successfully change the class labels to numeric for model classification
16/06/25	Columns as hexadecimal strings need to be converted to integers	Completed hexadecimal to integer conversions, changing the content to base-10 integers
17/06/25	Categorical features need to be one-hot encoded.	Managed to complete the encoding by creating new binary columns for the category for the model to read successfully
22/06/25	Applying Random Forest Classifier	Managed to import the RF classifier through the scikit learn libraries
22/06/25	Set up the model and set testing and training splits.	Correctly set code and made an 80% training and 20% testing split.
22/06/25	have 1st test with the model	The model was trained on the cleaned dataset and provided results, which were recorded.
23/06/25	Cleaned and tidied up code to look better	Completed
15/07/25	Research ways to augment the dataset after meeting with the supervisor.	Completed
17/07/25	Augmented dataset	Managed to duplicate the dataset and apply noise to help change samples, making it up to 87,000 samples - Completed
23/07/25	Complete several tests on the new dataset and the old datasets with different seeds to measure performance.	Successful test and managed to find the best seed with high results of classification.
29/07/25	Apply the model and set up a virtual real-time detection simulation, and measure the result.	Completed
4/08/25	3rd test on complete model to test classification results and overall model accuracy.	Completed.

SAMPLE THESIS

MSc PROJECT (COMP11024)

PROJECT PROCESS DOCUMENTATION NUMBER 2

Student: **Kai Johnson** **Supervisor:** **Dr. Althaff Irfan Cader Mohideen**

Meeting Number: **2** **Date/Time: 25/07/2025**

Agenda for meeting:

Discussing overall progress from the last meeting, providing feedback and other useful information.

Discussion of agenda items:

- Discussed the overall progress I was making and talked about the completion of the augmented dataset.
- Some general points to improve the clarity and presentation of the report.
- Happy with progress and for me to continue
- Talked about how to implement the model for the application.

Summary of agreed action plan:

Continue on the report and progress as normal, and research ways to simulate the model to show classifications.

Notes: meeting time 21 minutes, next meeting on the 19th August

SAMPLE THESIS

To-do list – Final report		
Date	**Details**	**Outcome**
20/08/25	Minor changes to report	Changes fine details such as the title, banner ID, and other small details
19/08/25	Fix up the Introduction	Made changes after meeting with the supervisor
19/08/25	Fix up, add diagrams, and add extra details to the literature review	Made changes after meeting with the supervisor
10/07/25	Complete Project Management	Completed
	Complete and finalise the research design and methodology.	Completed
23/07/25	Complete design chapter	Completed
27/07/25	Complete implementation Chapter	Completed
03/08/25	Complete testing chapter	Completed
10/08/25	Complete results chapter	Completed
18/08/25	Complete conclusion and recommendations chapter	Completed
18/08/25	Complete the critical self-evaluation chapter	Completed
19/08/2025	Complete references	Added all references used in the final report
19/08/2025	Bibliography	Added all extra references used to help with the project
19/08/2025	appendix	Added project specification form and process documentation.

SAMPLE THESIS

MSc PROJECT (COMP11024)

PROJECT PROCESS DOCUMENTATION NUMBER 3

Student: **Kai Johnson** **Supervisor:** **Dr. Althaff Irfan Cader Mohideen**

Meeting Number: **3** **Date/Time: 19/08/2025**

Agenda for meeting:

Discussing the final report, providing feedback and changes that can be made to make it an overall better report.

Discussion of agenda items:

- Changes to title page: Change title to efficacy over enhancing, add both banner IDs, change to MSC cyber security over Advanced computer security, and add supervisor name
- Update the library release form of the banner ID and title.
- Talked about abstract: make sure it has a problem, a solution and how it was achieved, no more than 300 words, no less than 200 words
- Fix the table of contents with chapter numbers, take out the current titles.
- Sign up to GitHub and upload the augmented dataset and all Python coding used so he has access.
- Add in figures and charts to the literature review discussions.
- Add in a chart of classified ransomware families.
- Add in charts of the dataset ransomware families and types of malware.

Summary of agreed action plan:

Fix the above problems in the main report, and it will be a well-rounded and good report.

Notes: Meeting time 1hr 4 minutes, discussed project presentation and what to expect, fix all issues above.

SAMPLE THESIS